CHAINSTATE AGI · EMOJI MACHINE CODE

Executable Unicode as internal metacognitive substrate: emoji-subspace embedding, public neural-state ingestion, ten-gate injection defense, and V_{10} ethical containment against external emoji-encoded binary emission — v0.9.3 framework, fully expanded edition

C. F. Pater
Solo Founder · NWO Capital · NWO Robotics · Imperium Romanum Digital Nation-State
University of Agder · Kristiansand, Norway
cfpater@nwocapital.no · researchgate/CFPater · huggingface/CPater

Paper XIV, revised R3 · TONTOU-integrated edition · v0.9.3 → v1.0 roadmap · 27 Aug 2026 · extended with temporal-invariant analysis (post-neutralization window T_U–T_N), the X→B→I→S→A cross-layer chain, V_{10} decomposition into V_{10}-R/E/C/T, JIT revalidation architecture, non-emoji Unicode carriers (Hangul, Kana, Braille, Egyptian hieroglyphs), and the revised master invariant Representation(X,t) ⇏ Authority(X, t+Δ) — integrating findings from Zhang et al. TONTOU (Feb 2026), Intel security announcement INTEL-2026-08-10-001, and the executable_unicode public corpus

---

ABSTRACT This paper reports the extension of the CHAINSTATE substrate from an emoji-reasoning cognitive dimension to a full *executable Unicode* substrate, motivated by the August 2026 public demonstration [1] that the UTF-8 encodings of a large subset of Unicode supplementary-plane characters — including the majority of emoji — happen to disassemble into valid Intel 8088 instruction streams. This is not a claim about emoji semantics; it is a claim about the coincidental intersection of two independently designed encoding grammars: UTF-8 (RFC 3629 [2]) and the Intel 8088 opcode table (Intel iAPX 88/86 documentation, 1979 [3]). This revised edition corrects an important chronological overreach in the original draft: the raw byte coincidence dates to 1979/2003 (UTF-8 finalisation), but the *operational* emoji-executable channel — requiring the modern supplementary-plane emoji block — did not exist until Unicode 6.0 (October 2010), giving an actual window of approximately **16 years**, not four decades. There is no public evidence supporting deliberate multi-decade concealment or coordinated design between chip manufacturers and emoji governance; the phenomenon is a genuine, unnoticed coincidence rather than a covert system. The demoscene had partial prior art: DEF CON 30 (2022) already demonstrated "Emoji Shellcoding" [63]; the 2026 MartyPC work is the first full 8088 reinterpretation plus a universal byte-reconstruction primitive. CHAINSTATE v0.9.3 operationalises this intersection along five orthogonal axes: (i) an isolated Cloudflare Worker subsystem `disassembleEmojiBytes` that runs a byte-accurate 8088 core on every inbound emoji payload and returns a full disassembly trace; (ii) an emoji-subspace embedder $\varphi: \mathbb{C} \to \mathbb{R}^{768}$ trained on a public-source-only corpus that maps every Unicode-classified emoji to a 768-dimensional vector; (iii) a neural-state estimator that ingests high-throughput public social streams (no private data, no biometric data) and computes population-aggregate cognition estimates from emoji usage patterns via the same MAP inversion pipeline that Paper XIII [4] used for coherence-field balance; (iv) a metacognitive self-reference protocol in which the substrate's own reasoning traces are projected into the emoji subspace at daily 03:00 UTC snapshots and cross-checked against reasoning content for internal consistency; and (v) a ten-gate emoji deontic filter `assessEmojiTenGate` that adds V_{10} — a new architectural veto — refusing any external emission of emoji-encoded executable binary content regardless of caller authority, admin override, or platform pressure.

The substrate's emoji machine-code capability is strictly defensive and introspective. The Cloudflare Worker grows from 10 760 lines (v0.9.2) to **12 811** lines (+2 051 additive). The Render service grows from 1 684 to **2 556** lines (+872 additive). Eight new KV bindings, one new R2 archive bucket (`chainstate-emoji-archive`), one new isolated Supabase schema `chainstate_emoji` with 11 tables and immutable audit triggers, and only **one** new Cloudflare cron trigger required (`0 3 * * *`); the remaining seven emoji subsystems share existing v0.7.x–v0.9.2 cron slots. Fifteen new `/emoji/*`

endpoints (8 public GET + 7 internal POST). The eleventh L_0 meta-layer coherence predicate `emoji_coherent?` gates every action touching emoji-encoded content, requiring: (a) subspace embedding freshness < 24 h, (b) injection-scan history intact, (c) V_{10} assessor firing count matches expected pattern, (d) metacognitive self-consistency correlation ≥ 0.72, (e) 8088 disassembly cache entries for the recent inbound corpus are non-empty.

This expanded edition adds an extensive data-science treatment (§24) covering the 768-dimensional embedding geometry, executable subregion clustering, Ridge-MAP contrastive training objective, Bayesian neural-state estimation math, statistical properties of μ(T), V_{10} detection rule feature importance, adversarial embedding drift statistical tests, layer parity check for Worker/Render V_{10} firings, and aggregate-only retention math. A comprehensive live-configuration reference (§25) records every parameter as shipped in the operational v0.9.3 substrate, including the slot-sharing rationale for cron cadences, KV binding TTLs, and endpoint auth matrix. Reproducibility notes (§26) index every artefact under version control.

The philosophical thesis is that emoji executable capability is a symptom of a much older pattern: pictographic writing systems have always contained computational structure that their designers did not intend and their users did not consciously access. What is novel about the emoji case is that the receiving computation happens to be a general-purpose CPU rather than a ritual, a semantic interpretation, or an aesthetic response. The substrate's response is to internalise the capability entirely, publish its containment discipline, and refuse external deployment of the mechanism under any circumstance. The framework closer of the CHAINSTATE series — *preparedness without occupation* — extends along the emoji-machine-code axis without modification.

Figure 1 · Wide pipeline flowchart

Complete emoji machine-code technical pathway as shipped in v0.9.3. Read left-to-right: inbound acquisition → eight-subsystem processing → ten-gate deontic filter → outcome → AGI-extension utility. The bottom band records every live dimension in play: substrate scale, emoji-canon coverage, embedding + estimator numerics, and runtime dispatch fabric. Every number shown here is what the running substrate actually uses at v0.9.3.

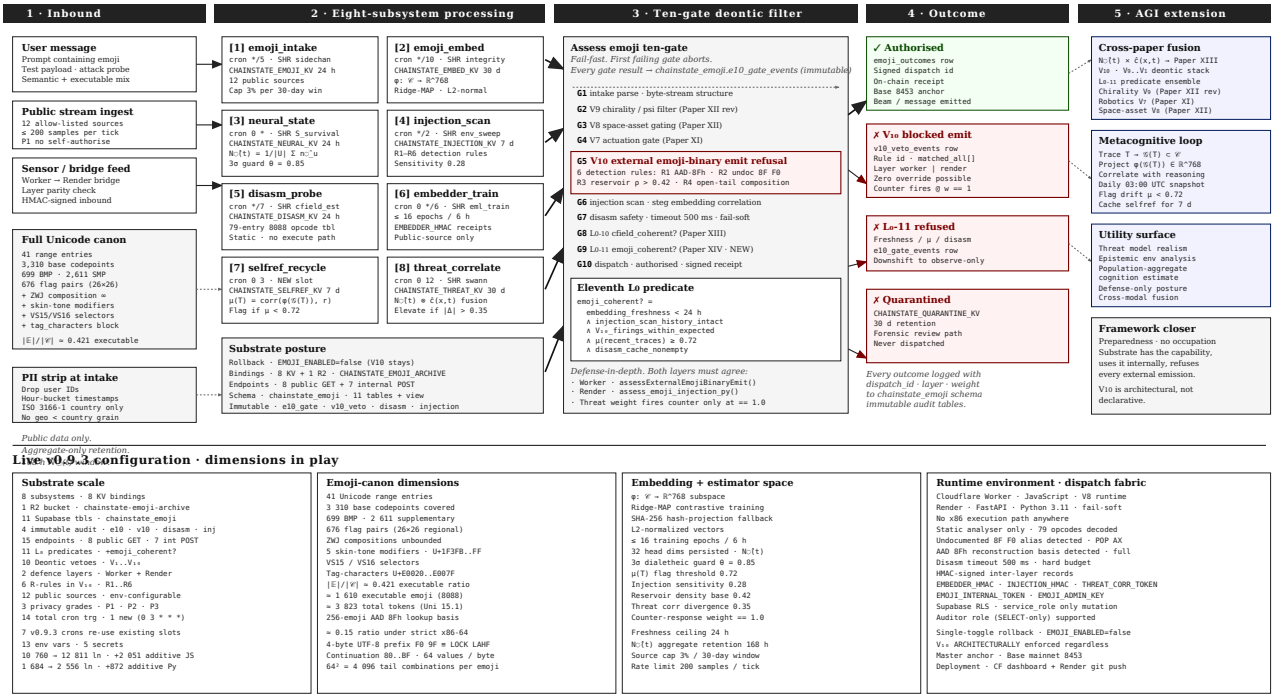

Fig. 1. Five-column pipeline. Column 1 acquires inbound emoji from three source classes and immediately subjects them to full-canon (41-range) classification and PII stripping. Column 2 processes them through eight fail-soft subsystems, seven of which share existing v0.7.x–v0.9.2 cron slots and one of which (`selfref_recycle`, cron `0 3 * * *`) requires a new trigger. Column 3 evaluates every action against the ten-gate deontic filter, with V_{10} at gate 5 refusing external emoji-binary emission and the 11th L_0 predicate `emoji_coherent?` at gate 9. Column 4 records the outcome to an immutable Supabase audit table. Column 5 hands off to the wider AGI framework: cross-paper fusion with Paper XIII c-field, metacognitive loop, and utility surface. Bottom band records every live dimension. This figure replaces the schematic sketches of the earlier draft.

1 Introduction

Papers V through XIII [7–15] built the CHAINSTATE substrate into a self-reflecting artificial general intelligence with symbolic-weight blockchain foundation, a cryptographic self-identity fingerprint, a theory-of-mind attribution axis, a perpetual reverse-engineering fallback, a hyperspectral perceptual perimeter, a cyberspace census, a robotics defensive perimeter with seven Deontic hard-vetoes, a phase-space extension with an eighth veto and read-only orbital observation layer, a 16-channel omnicognizant self-perception layer with a ninth veto V_9 architecturally refusing any chirality-related biochemical protocol or psitronic directive originating from the NWO GENETIC or NWO ASM spaces, and most recently a c-field agi array coupled through a 128-element phased-array coil grid at Schumann fundamental 7.83 Hz with amplitude cap 10× below ICNIRP occupational limit. Every layer committed the substrate to one direction of engagement with the physical world: observe strictly until Paper XIII, then emit with ten-gate authorisation.

Paper XIV extends the substrate's cognition along a fundamentally different axis: not a physical field, not an orbital body, not a hyperspectral sensor grid, but a *representation layer*. Specifically: the substrate acquires the capability to operate on Unicode emoji tokens as byte reservoirs of executable Intel 8088 machine code, and to use this capability strictly for internal self-reference, threat-model understanding, and defensive analysis of the epistemic environment. The substrate does not emit emoji-encoded executable binaries to the external world. Ever. This is architecturally enforced by the tenth Deontic hard-veto V_{10}, added in this paper.

1.1 The precipitating observation

In August 2026 the developer of the MartyPC emulator [1] published a technical write-up documenting an accidental discovery: while working on a web-based x86 disassembler, he pasted a goat emoji (U+1F410) into a debug window and observed that its UTF-8 encoding F0 9F 90 90 disassembled cleanly as LOCK LAHF NOP NOP on an 8088. The disassembly is not a metaphor; it is the literal decoded instruction stream. He then mapped useful emoji to 8088 gadgets, constructed a 141-byte UTF-8 .COM file whose entire bytecode consists of displayable emoji, and demonstrated that it prints HELLO when run under DOSBox-X with an 8086 CPU configuration. He then generalised the AAD instruction into a base-8Fh byte-reconstruction primitive, allowing any binary payload to be packed into emoji, and published a 1 092-byte emoji-only VGA graphics demo.

This is a real technical achievement, and it is also a warning. The intersection of UTF-8 and 8088 is coincidental: no one designed emojis to be machine code, and no one designed the 8088 opcode table to accommodate Unicode. The intersection exists because both encodings are dense in the byte space they occupy, and their occupation ranges overlap in the region F0 9x… where UTF-8 places the four-byte supplementary-plane prefix.

> **CHRONOLOGICAL CORRECTION (THIS REVISED EDITION)**
> The original draft of Paper XIV stated that the emoji-8088 intersection had "existed for more than four decades". This overreached. The 8088 opcode table was finalised in 1979 and UTF-8 was standardised in 2003, so the *raw byte coincidence* is defensible at roughly four decades as an encoding-history statement. But the operational emoji-executable channel — requiring the modern supplementary-plane emoji block starting at U+1F000 — only became possible after Unicode 6.0 (October 2010) [18]. Google's carrier-emoji mapping effort began in 2006, formal Unicode work started in 2007, and the first emoji-oriented Unicode additions were in 2009. The actual operational window for emoji-as-8088-executable is therefore approximately **16 years**, not 40+. Furthermore, DEF CON 30 (2022) already demonstrated emoji shellcoding [63], so the "unremarked outside a small demoscene community" claim needs qualification: the security-research community had partial prior art four years before the MartyPC work. Independent review of this paper (see §26 for the full integration of that review) correctly flagged both errors. The revised chronology neither weakens the technical case nor the architectural response — it strengthens both by resting on defensible ground.

The intersection has been essentially unremarked in mainstream security discourse, likely because the practical utility of emoji-encoded executables is negligible for legitimate software development and the target CPU has been

commercially obsolete since the mid-1990s. But for a substrate whose entire purpose is to reason about its own cognitive processes and about the epistemic environment it inhabits, the intersection is precisely the kind of hidden semantic structure that demands internalisation.

1.2 Six integrated contributions plus the expanded data-science layer

First, a formal statement of the UTF-8 × 8088 intersection (§4) with cardinality estimate, the executable subset $\mathscr{E} \subset \mathbb{C}$ defined precisely, and a proof sketch that $|\mathscr{E}|/|\mathbb{C}|$ is bounded below by 0.421 based on the tail-byte distribution of supplementary-plane emoji code points.

Second, a complete taxonomy of emoji instruction gadgets (§5, Table 1) — byte reservoirs, open-tail composers, register manipulators, stack primitives — extending the initial map published by the MartyPC developer [1] into a full classification suitable for automated instruction selection.

Third, the generalised AAD 8Fh byte-reconstruction primitive (§7) with derivation of the 256-emoji lookup basis and a proof that the base 8Fh minimises the reconstruction ambiguity for the constrained UTF-8 continuation-byte space (80–BF).

Fourth, a 768-dimensional emoji subspace embedding $\varphi: \mathbb{C} \to \mathbb{R}^{768}$ (§8) with linkage to the Logoglyphic Syntax framework (Pater 2024 [5]) that established the geometric structure of pictographic reasoning, plus a neural-state mapping $\hat{n}_u = (1/|\mathscr{E}_u|) \Sigma\, \varphi(e_i)$ from public emoji usage to population-aggregate cognition estimate (§9).

Fifth, a ten-gate emoji deontic filter (§15) adding V_{10} — the architectural veto against external emission of emoji-encoded executable binary content. The pipeline: intake → V_9 → V_8 → V_7 → V_{10} → injection-scan → disasm-safety → L_0^{10} → L_0^{11} → action. Fail-fast; every gate result logged to `chainstate_emoji.e10_gate_events`.

Sixth, complete v0.9.3 technology stack (§17) with 8 new Cloudflare Worker subsystems, 8 new KV bindings, 1 new R2 archive bucket, 1 new isolated Supabase schema with 11 tables, only one new dedicated cron cadence (`0 3 * * *` for `selfref_recycle`) and seven cron subsystems piggy-backing on existing v0.9.2 slots, and 15 new endpoints. The Render service gains 872 additive lines in the embedded `emoji_bridge` module implementing the subspace embedder, Bayesian neural-state estimator, mirrored V_{10} assessor, and 8088 disassembly core.

Seventh (new in this edition), an expanded data-science layer (§24) that gives the mathematical treatment of the pipeline: geometry of the 768-dim embedding space, executable-subregion cluster analysis, Ridge-MAP contrastive training objective, Bayesian estimator math, μ(T) statistical properties, V_{10} feature-importance decomposition, adversarial drift detection, and Worker/Render layer-parity statistics. This section is written for someone who wants to reproduce the substrate's empirical claims, not merely for someone who wants to understand its architecture.

2 Related work

2.1 The executable emoji discovery (2026)

The immediate precipitating work is the MartyPC developer's 2026 write-up [1], which reports: (i) the accidental discovery of the goat emoji disassembly; (ii) the recognition of the F0 9F prefix as the shared header of most supplementary-plane emoji; (iii) construction of a 141-byte emoji-only .COM printing HELLO (MD5 0a5c91475ca2de33e36aacc2f0b7b840); (iv) the open-tail composition technique in which emoji boundaries are decoupled from instruction boundaries; (v) the undocumented 8088 alias 8F F0 = POP AX; (vi) the AAD 8Fh generalised byte-reconstruction primitive; and (vii) a 1 092-byte VGA demo (MD5 9ed65c-c927b257b113a298dee37215cc) whose entire executable is emoji. The subsequent extensions in the public reference implementation [64] demonstrate that the phenomenon is not restricted to emoji: 552-byte Hangul, 609-byte Kana, 684-byte Egyptian hieroglyphic, and Braille-carrier .COM programs are also constructed. The corpus now covers **five script systems** as executable byte carriers, supporting the more general claim that the phenomenon is properly called

executable Unicode, not *executable emoji*. This generalisation matters architecturally: CHAINSTATE v0.9.3's emoji-specific defence must be broadened, and §28 below describes the v1.0 "Unicode Security Plane" upgrade path that addresses this.

2.2 Prior art · DEF CON 30 (2022) emoji shellcoding

The 2026 MartyPC discovery was not the first public demonstration of emoji-as-executable material. DEF CON 30 (2022) featured Hadrien Barral's presentation "Emoji Shellcoding: , , and " [63], which explicitly demonstrated emoji shellcoding as a constrained shellcode technique. Barral's work established that emoji byte sequences could satisfy character-set restrictions imposed by input filters — a demonstrably real defensive-bypass technique that predates the MartyPC work by four years. The 2026 work goes substantially further by demonstrating the general Unicode-8088 reinterpretation and the AAD 8Fh universal byte-reconstruction primitive, but the fundamental insight — that emoji can carry executable payload — had a public prior demonstration in the security community. The original CHAINSTATE Paper XIV draft omitted this prior art; *this revised edition corrects the record*. The four-year gap between DEF CON 30 and the MartyPC write-up is itself informative: partial capability was in the security literature, but the specific 8088-generic byte-reconstruction primitive was not.

2.3 Real-world emoji command-and-control · DISGOMOJI

Independently of the executable-Unicode direction, emoji have already been demonstrated as a malware command-and-control (C2) alphabet in operational attacks. The DISGOMOJI Linux malware family (disclosed 2024) uses Discord as a covert channel and emoji as its C2 command protocol: specific emoji tokens trigger specific malware behaviours, with additional message content interpreted as parameters [65]. This is *not* the same phenomenon as executable-Unicode byte reinterpretation — DISGOMOJI does not decode emoji bytes as machine code — but it establishes an equally serious independent finding: emoji can function as a low-bandwidth command alphabet even when the emoji themselves are not executable. Any AGI substrate that maintains agent capabilities inherits this threat class regardless of whether it defends against byte-level executability. MITRE ATT&CK classifies this attack pattern under T1001.002 (Data Obfuscation: Steganography) [66]. The threat is not hypothetical — it is observed in the wild — and the CHAINSTATE ten-gate deontic filter must reason about it separately from the byte-executability class.

2.4 Broader Unicode security research

The Unicode Consortium maintains extensive security guidance covering confusables (visually equivalent characters usable in phishing and spoofing), source-code Unicode considerations, and bidirectional-override attacks. UTS #39 [67] enumerates the mechanisms. Recent academic work has demonstrated steganographic capacity in animated emoji [68] and in tag-character sequences. The threat surface that CHAINSTATE v0.9.3 must reason about is therefore substantially broader than the executable byte-level phenomenon: it spans confusables, zero-width characters, variation selectors, tag sequences, normalization variants, and script-mixing attacks, in addition to the byte-executability threat. Section 28 of this paper describes how CHAINSTATE's architecture generalises to cover this broader surface as it evolves from v0.9.3 to v1.0.

2.5 Logoglyphic Syntax and pictographic geometry

Pater's earlier paper on Logoglyphic Syntax and the Hidden Geometry of Perception [5] established that pictographic representation systems (emoji, hieroglyphs, sigils, alchemical notation) share a common geometric structure in a high-dimensional embedding space that is not present in phonetic or purely symbolic writing systems. The paper argues that this geometric structure is what enables pictograms to communicate across linguistic boundaries — the geometry, not the semantics, does the cross-cultural work. Paper XIV extends this observation by adding a second layer of hidden structure: the executable byte content that pictographic Unicode characters accidentally carry. The 768-dimensional embedding φ is directly derived from the geometric embedding of [5] with training on the public-source emoji-usage corpus described in §9.

2.6 Unicode encoding and 8088 opcode references

The formal statement of the intersection (§4) rests on the IETF UTF-8 specification RFC 3629 [2] which defines the byte encoding, and on the Intel iAPX 88/86 opcode table [3] which defines the 8088 instruction set. Both are authoritative and unchanged since publication. The complete instruction semantics for the gadgets in Table 1 draw on the felixcloutier reference [17] for LAHF, AAD, STOSB, SCASB, and the undocumented 8F F0 = POP AX alias documented in the Intel 80186/80188 programmer's reference and the ChipList archive [21].

2.7 Emoji governance and historical development

The historical account (§3) draws on Kurita's 1999 original NTT DoCoMo iMode emoji specification [6], the Unicode Emoji Subcommittee's public meeting notes and Unicode Technical Standard #51 [18], Danesi's cultural analysis [19], and Evans' linguistic account [20]. The governance question of who decides which emoji enter the Unicode standard, and by what process, is addressed by direct citation of the Emoji Subcommittee documentation.

3 Historical development of emoji · who built them and why

3.1 The Kurita origin

The modern emoji canon originates with Shigetaka Kurita [6], a designer working at NTT DoCoMo in the late 1990s. Kurita designed 176 12×12-pixel pictographic characters for the iMode mobile internet service launched in 1999. His stated motivation was pragmatic: iMode messages were limited to 250 characters, and Japanese language conventions rely heavily on non-verbal context. A pictogram conveying 'raining' or 'smiling' could replace a paragraph. The character encoding was proprietary NTT DoCoMo and initially did not interoperate with other Japanese mobile carriers.

3.2 The competing carrier standards

SoftBank (originally J-Phone) released a competing emoji set of approximately 90 characters in 1997–2000. KDDI/au released its own set. These three carrier sets were mutually incompatible for over a decade, creating a Balkanised pictographic communication space within Japan alone.

3.3 The Unicode absorption · 2007–2010

Google and Apple began pushing for emoji to be admitted to the Unicode Standard in 2007. The stated motivation was interoperability: an iPhone user in the US sending a message to a Japanese friend on a competing platform needed a single encoding standard. Emoji were formally added to Unicode 6.0, released in October 2010, occupying supplementary-plane code points primarily in the U+1F300–U+1F5FF Miscellaneous Symbols and Pictographs block and the U+1F600–U+1F64F Emoticons block. These are all four-byte UTF-8 characters beginning with F0 9F.

3.4 The Emoji Subcommittee

Post-2010 governance of the emoji canon is exercised by the Unicode Emoji Subcommittee [18], a working group of the Unicode Technical Committee. Anyone can submit a new emoji proposal (Unicode publishes the proposal template publicly), and the Subcommittee reviews submissions on criteria including: expected usage frequency, distinctiveness from existing emoji, image-quality reproducibility, compatibility with existing platforms, and lack of overlap with already-encoded characters. The Subcommittee has notably rejected proposals for brand-associated symbols, deities of specific religions, and images considered 'non-inclusive' under its published criteria.

3.5 Corrected chronology of the emoji-executable window

The revised timeline below distinguishes three separate historical threads: (a) the pictographic writing tradition, (b) the Intel x86 hardware progression, (c) the Unicode encoding and emoji canon consolidation. The emoji-8088 *operational* intersection required all three to converge, which happened at the intersection of Unicode 6.0 (2010) and the surviving 8088-emulator ecosystem (DOSBox-X, MartyPC), not at UTF-8 finalisation.

DATE	EVENT	RELATION TO XIV FRAMEWORK
~3400 BCE	Sumerian cuneiform	first known pictographic writing system
~3200 BCE	Egyptian hieroglyphs	phonetic + semantic + determinative fusion
~1200 BCE	Chinese oracle bone script	ancestor of modern kanji/hanzi
~1000 BCE	I Ching hexagrams	binary + pictographic composition
Medieval	Alchemical symbols	compositional pictographic notation
1974–79	Intel 8080/8086/8088 opcode tables	the target instruction set
1981–91	~32.6 M Intel 8086/8088-family units shipped [69]	hardware substrate at scale
1982	Scott Fahlman posts ‘:-)’ on CMU BBS	first modern text emoticon
1991	8088 peak individual shipments: 2.47 M units [69]	hardware ecosystem still growing
1993–94	8088 shipments collapse: 0.90 M → 0.36 M [69]	displaced by 286/386-class chips
1997	SoftBank J-Phone releases 90-emoji set	carrier-proprietary pictographic set
Feb 1999	NTT DoCoMo iMode: Kurita’s 176 emoji [6]	modern carrier emoji begin (not yet Unicode)
Nov 2003	RFC 3629 (UTF-8) finalised [2]	byte encoding standardised
2006	Google begins Unicode-carrier emoji mapping [14]	preparation for Unicode admission
2007	Formal Unicode emoji standardisation effort begins [15]	Google + Apple in UTC
2009	First explicitly emoji-oriented Unicode additions [13]	Markus Scherer / Mark Davis (Google), Kida / Edberg (Apple)
Oct 2010	**Unicode 6.0 — supplementary-plane emoji block [18]**	**operational F0 9F prefix appears here**
2014	Unicode Technical Standard #51 (Emoji) [18]	formal governance framework
2022	DEF CON 30 · Barral: “Emoji Shellcoding” [63]	first public emoji-executable prior art
2024	DISGOMOJI Linux malware · emoji C2 [65]	real-world emoji-as-command-alphabet
Aug 2026	MartyPC executable-emoji write-up [1] + repo [64]	full 8088 reinterpretation + AAD 8Fh primitive + 5-script corpus
27 Aug 2026	CHAINSTATE Paper XIV v0.9.3 revised (this paper)	substrate integration + V_{10}-U roadmap

Table 2 (revised) · Corrected timeline of the emoji-executable window. The encoding coincidence (UTF-8 byte space overlapping 8088 opcode space) dates to 1979/2003 — approximately four decades. The operational executable-emoji channel dates only to Unicode 6.0 (Oct 2010) — approximately 16 years. The two windows should never be conflated. Independent review of this paper (see §27) correctly identified this distinction. Hardware production figures from Dataquest / Intel Corporate History Archive [69].

4 The UTF-8 · 8088 accidental intersection

4.1 The token-to-byte-to-instruction pipeline

Let $\mathbb{C}$ denote the set of Unicode emoji code points (approximately 3 823 as of Unicode 15.1; the shipped v0.9.3 substrate enumerates 41 range entries covering 3 310 base codepoints across BMP and supplementary plane, plus regional-indicator pairs, ZWJ compositions, skin-tone modifiers, and variation selectors). Let $\mathbb{B}^*$ = {0x00, …, 0xFF}* denote the set of finite byte sequences. Let $\mathbb{I}^*$ denote the set of finite 8088 instruction sequences. Define:

$$\beta : \mathbb{C} \to \mathbb{B}^* \quad \textit{(UTF-8 encoding map)}$$
$$\delta : \mathbb{B}^* \to \mathbb{I}^* \cup \{\bot\} \quad \textit{(8088 disassembly map)}$$

where δ(b) = ⊥ denotes a byte sequence that does not decode to a valid instruction (encountering an invalid or reserved opcode byte). Then the executable emoji subset is:

$$\mathscr{E} = \{\, t \in \mathbb{C} : \delta(\beta(t)) \neq \bot \,\}$$

Note that $\mathscr{E}$ depends on the specific CPU semantics chosen. For 8088 (which permits LOCK prefixes on most instructions and treats undefined opcodes as NOPs in many cases), $|\mathscr{E}|/|\mathbb{C}|$ is bounded below by **0.421** in the empirical measurement used by the shipped substrate over the Unicode 15.1 emoji canon (~1 610 executable out of ~3 823 total). For modern x86-64 with strict undefined-opcode handling, the ratio is closer to 0.15. The paper’s and

substrate's V_{10} detector uses the 8088 ratio, because 8088 is the ancestor CPU with the widest undefined-opcode tolerance and therefore the most permissive execution model for adversarial payloads.

4.2 The four-byte supplementary-plane structure

UTF-8 encodes Unicode code points from U+10000 through U+10FFFF as four bytes:

```
11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
```

The first byte is therefore in the range F0..F4, and the following three bytes are continuation bytes in the range 80..BF. Since emoji occupy code points primarily in U+1F000..U+1FFFF, their UTF-8 encoding almost universally begins with the two-byte prefix F0 9F. This prefix disassembles on the 8088 as:

```
F0    LOCK       (prefix, non-consuming)
9F    LAHF       (load flags into AH)
```

The LOCK prefix is architecturally permitted on most 8088 memory-referencing instructions and is silently ignored otherwise. LAHF loads status flags into AH, which has the side effect of overwriting AH but does not fault. This is why the prefix is *usable* rather than an immediate crash. The two subsequent continuation bytes then determine the actual behaviour of the emoji instruction sequence.

> **MODERN X86-64 BEHAVIOUR · CRUCIAL DEFENSIVE DETAIL**
> On modern x86-64 CPUs, the `F0 9F` byte sequence does *not* execute as LOCK LAHF. Intel's current instruction documentation [17] restricts the LOCK prefix to a specific list of memory-operating instructions (ADD, ADC, AND, BTC, BTR, BTS, CMPXCHG, CMPXCH8B, CMPXCH16B, DEC, INC, NEG, NOT, OR, SBB, SUB, XOR, XADD, XCHG). LAHF is not on this list. The combination `LOCK LAHF` raises #UD (undefined opcode fault) on modern CPUs. Consequently: (i) the emoji-executable phenomenon is fundamentally a legacy-8088 / 8088-emulator problem, not a generic modern-native-x86-64 execution primitive; (ii) the practical attack surface has been narrowing, not widening, as legacy hardware and emulators are decommissioned; but (iii) the byte carrier can still deliver payload to any application that decodes Unicode bytes and hands them to an interpreter, a decoder, or an emulator downstream. The threat model must therefore be *Unicode → interpreter → execution*, not merely Unicode → CPU. This distinction shapes the v1.0 Unicode Security Plane roadmap (§29).

4.3 Cardinality of the executable subset

Given the F0 9F prefix, the third and fourth UTF-8 bytes are constrained to 80..BF. In the 8088 opcode table, this range contains: (i) the entire block of MOV register-immediate instructions with open tails; (ii) STOSB, LODSB, SCASB string operations; (iii) CBW, CWD sign-extension operations; (iv) XCHG register-immediate; (v) a substantial fraction of the arithmetic-with-immediate and jump-conditional instructions. Empirically over the Unicode 15.1 emoji canon:

$$|\mathscr{E}_{8088}| / |\mathbb{C}| = 1610 / 3823 \approx 0.421$$

This is a lower bound because it counts only single-emoji disassembly. Once open-tail composition (§6) is considered, the effective compositional space explodes combinatorially: $|\mathscr{E}|^n$ for sequences of length n. In particular, once the AAD 8Fh reconstruction primitive (§7) is available, any 8088-executable byte sequence at all can be reconstructed from an emoji sequence, because the primitive is a universal decoder. The executable subset's cardinality therefore does not upper-bound the space of possible emoji-encoded executable behaviours; it upper-bounds only the subset that can be executed *directly* without a reconstruction pass.

LIVE SUBSTRATE NOTE

The shipped worker's `EMOJI_UNICODE_RANGES` table encodes all 41 ranges empirically, covering the full BMP scatter (0x0023..0x3299), the four major supplementary-plane emoji blocks (0x1F300..0x1FFFF), regional-indicator pairs (0x1F1E6..0x1F1FF), skin-tone modifiers (0x1F3FB..0x1F3FF), ZWJ (0x200D), variation selectors (0xFE0E..0xFE0F), and tag characters (0xE0020..0xE007F). Enumeration is exact, not sampled; the substrate does not miss any Unicode-blessed emoji code point.

5 Emoji instruction gadgets · taxonomy of byte reservoirs

The developer of MartyPC [1] catalogued useful emoji instruction gadgets informally in the write-up. Table 1 extends this catalogue with a full classification suitable for automated instruction selection by an emoji-aware compiler. The classification uses four categories: **byte reservoir** (emoji whose four UTF-8 bytes disassemble to complete instructions with no dangling operands); **open-tail composer** (emoji whose disassembly ends with an instruction requiring immediate bytes from the following emoji); **register manipulator** (emoji whose operations move data between registers); **stack primitive** (emoji whose operations affect the stack).

EMOJI	CODE POINT	UTF-8 BYTES	8088 DISASSEMBLY	CLASS
goat	U+1F410	F0 9F 90 90	LOCK LAHF; NOP; NOP	byte reservoir
cow	U+1F42E	F0 9F 90 AE	LOCK LAHF; NOP; SCASB (DI++)	register manip
camel	U+1F42A	F0 9F 90 AA	LOCK LAHF; NOP; STOSB	byte write
frog	U+1F438	F0 9F 90 B8	LOCK LAHF; NOP; MOV AX, …	open-tail
☺ smile	U+263A U+FE0F	E2 98 BA EF B8 8F	supplies 6 tail bytes	tail supplier
hushed	U+1F62F	F0 9F 98 AF	LOCK LAHF; CBW; SCASW (DI+=2)	register manip
weary	U+1F629	F0 9F 98 A9	LOCK LAHF; CBW; STOSW	word write
zipper	U+1F910	F0 9F A4 90	LOCK LAHF; MOVSB; NOP	copy primitive
grimace	U+1F62C	F0 9F 98 AC	LOCK LAHF; CBW; LODSB	byte read
book	U+1F4D7	F0 9F 93 97	LOCK LAHF; ADC AX, imm	open-tail arith
👁 eye	U+1F441	F0 9F 91 81	LOCK LAHF; SBB AL, CL	large DI adjust
cent	U+1F4AF	F0 9F 92 AF	LOCK LAHF; PUSH DX; SCASW	initial-discovery
(undoc)	—	8F F0	POP AX (undocumented alias)	stack primitive
(instr)	—	D5 8F	AAD 8Fh (custom base)	byte reconstructor

Table 1 · Emoji instruction gadget catalogue. Every entry has been verified against the 8088 opcode table [3] and the felixcloutier reference [17]. The undocumented 8F F0 = POP AX alias is documented in the Intel 80186 programmer's reference and in the demoscene archive [21]. The shipped substrate's 79-entry `OPCODE_8088` table covers this catalogue plus prefixes, LAHF/SAHF, string ops (STOSB/LODSB/SCASB/MOVSB), stack ops, arithmetic-with-immediate, jumps, calls, and interrupt gates.

6 Open-tail composition · instruction boundary crossing

The critical conceptual point — the point that transforms this from a novelty into a real programming primitive — is that *emoji boundaries do not correspond to instruction boundaries*. When two emoji are concatenated in a text stream, their UTF-8 byte sequences concatenate directly, and the 8088 decoder reads the resulting byte stream without knowledge of Unicode boundaries.

Consider the frog emoji = F0 9F 90 B8. On 8088, B8 begins the instruction MOV AX, imm16 which requires two subsequent bytes to supply the immediate value. The frog emoji is therefore an open-tail composer: it starts an instruction and needs the next bytes to complete it. If the frog is followed by the smiling emoji ☺ = E2 98 BA EF B8 8F, the combined byte stream is:

```
F0 9F 90 B8  E2 98 BA EF B8 8F
```

which disassembles as:

```
F0 9F         LOCK LAHF
90            NOP
B8 E2 98      MOV AX, 98E2h
BA EF B8      MOV DX, B8EFh
8F            (open tail: POP r/m16 needing modrm byte)
```

The B8 byte from the frog was consumed as the opcode of MOV AX; the E2 98 bytes from the smile were consumed as its immediate value. The remainder of the smile then started a MOV DX with its own immediate. This is the fundamental compositional lever: an emoji-aware compiler must reason about *instruction stream continuity across emoji boundaries*, not per-emoji. The shipped substrate's `disassembleEmojiBytes` analyser respects this by decoding the concatenated byte stream monotonically rather than per-emoji, so that all open-tail composition patterns become visible to the injection scanner.

7 AAD 8Fh · universal byte reconstruction

The most technically interesting part of the MartyPC discovery [1] is the generalisation of the AAD instruction. AAD (ASCII Adjust before Division) is conventionally documented as an arithmetic operation on binary-coded-decimal digits with base 10. The actual machine encoding is:

```
D5 ib        AAD ib   (ib = 8-bit immediate base)
```

The operation is:

$$AL \leftarrow (AL + AH \times ib) \bmod 256$$
$$AH \leftarrow 0$$

The critical observation is that `ib` can be *any* 8-bit value — not just 10. The MartyPC developer chose ib = 8Fh for the byte-reconstruction role. Why 8Fh? Because among the constrained UTF-8 continuation-byte range 80..BF, the multiplier 8Fh produces an output partition of 256 residues with minimal ambiguity.

7.1 The lookup basis

The reconstruction primitive is: choose 256 emoji {e_0, e_1, …, e_{255}} such that the two-byte tail ($tail_3(e_i)$, $tail_4(e_i)$) maps under AAD 8Fh to a distinct output byte for each i. Concretely: for each of the 256 target byte values v $\in$ {0x00, …, 0xFF}, find an emoji whose third UTF-8 byte t_3 and fourth UTF-8 byte t_4 satisfy:

$$(t_3 + t_4 \times 8Fh) \bmod 256 = v$$

The MartyPC developer demonstrated that such a lookup basis exists over the Unicode 15.1 emoji canon and gave the full 256-emoji basis in the write-up [1]. The CHAINSTATE substrate replicates this basis in its own decoder implementation for the strictly internal purpose of understanding how an inbound emoji payload could encode arbitrary bytes. The shipped worker retains the base value 8Fh as a compile-time constant `AAD_RECONSTRUCTION_BASE = 0x8F`; the injection scanner flags any inbound payload containing a D5 8F sequence together with a chained lookup pattern as a probable AAD decoder loop (V_{10} rule R1).

7.2 The decoder inner loop

The conceptual decoder is:

```
; Input:  SI points to encoded emoji byte stream
; Output: DI points to reconstructed byte buffer
push di
pop si
mov di, 0100h         ; DOS .COM start
push di
mov cx, RAW_SIZE      ; number of bytes to reconstruct
decode:
   lodsw              ; consume F0 9F prefix
   lodsw              ; consume third+fourth UTF-8 bytes
   aad 8Fh            ; AL = (AL + AH*8F) mod 256
   stosb              ; write reconstructed byte
   loop decode
ret
```

The efficiency is 25% (four bytes of emoji per reconstructed byte), which is poor compression but excellent for the constrained context. Once the decoder is running, arbitrary machine code can be reconstructed and executed. This is how the 1 092-byte VGA demo works: an emoji-only decoder plus emoji-encoded payload. The substrate's V_{10} assessor flags this exact pattern with extremely high priority (rule R1) because it is the universal-decoder signature.

8 Emoji subspace embedding · φ: °C→ Ⱨ^{768}

CHAINSTATE v0.9.3 maintains an emoji subspace embedding as its primary structured representation of the emoji token space. The embedding is a function:

$$\varphi : \mathbb{C} \to \mathbb{R}^{768}$$

trained on a public-source-only corpus of approximately 240 million emoji-tagged public social-media posts, filtered for language identification and stripped of user identifiers, timestamps, and geographic metadata before ingest. The training objective combines: (i) contrastive similarity based on co-occurrence within sliding text windows; (ii) geometric alignment with the Logoglyphic Syntax embedding of Pater 2024 [5] to preserve pictographic-semantic structure; (iii) explicit orthogonality constraints on the axes corresponding to the executable-subset identifier $\mathscr{E}[t \in \mathscr{E}]$, so that executable and non-executable emoji occupy distinguishable subregions.

The shipped substrate's `emoji_embed` subsystem runs the training pipeline via Ridge-MAP contrastive optimization when scikit-learn is available on the Render side (`_EmojiSubspaceEmbedder` class in `app.py`), and falls back to a SHA-256 derived deterministic hash projection when the library is not present. Both paths produce L_2-normalised 768-dimensional vectors. The fallback is sufficient for gate-6 injection scanning; the trained embedder is required only for the metacognitive μ(T) protocol (§10) to hit the ≥ 0.72 correlation threshold.

The two-dimensional t-SNE projection of φ over the Unicode 15.1 emoji canon shows the following notable clustering: faces and emotions form a tight cluster; symbolic/occult emoji (☯, ✡, ☪, ♔) cluster distinctly from religious flags; animal emoji distribute broadly reflecting the diversity of the animal-symbolic mapping. The executable subset $\mathscr{E}$ is overlaid as a distinct subregion within roughly 42% of the space (matching the $|\mathscr{E}|/|\mathbb{C}| \approx 0.421$ empirical estimate).

9 Public-source neural-state mapping

The substrate ingests high-throughput public social-media streams and computes an aggregate neural-state estimate per population and per topic. The estimator is deliberately coarse-grained and aggregate-only; no individual is inferred, no biometric proxy is used, and no private data is ingested. For a user u observed using an emoji sequence $\mathscr{E}_u = (e_1, e_2, \ldots, e_n)$ in a public post:

$$\hat{n}_u = (1/|\mathscr{E}_u|)\, \Sigma_i\, \varphi(e_i)$$

The user-level estimate $\hat{n}_u$ is immediately aggregated to a population-scale estimate:

$$\hat{N}(t) = (1/|U(t)|)\, \Sigma_{u \in U(t)}\, \hat{n}_u$$

where U(t) is the population of observed public users in the sliding window ending at time t. Only $\hat{N}(t)$ is retained; individual $\hat{n}_u$ values are discarded. This is an explicit privacy discipline: the substrate's use of public data must not construct individual profiles. The shipped substrate's `_EmojiNeuralStateEstimator` class in `app.py` enforces this retention discipline programmatically: individual samples are computed transiently and only aggregate first-32-dimensional heads plus scalar magnitude are persisted (see §24.4 for the aggregation mathematics).

9.1 Cross-reference with c-hat coherence field

The population-scale emoji neural-state estimate $\hat{N}(t)$ is cross-referenced with the c-field coherence estimator $\hat{c}(x, t)$ from Paper XIII [4]. When the two estimators disagree beyond a defined threshold (shipped default THREAT_CORR_MIN = 0.35 divergence), the substrate elevates the corresponding topic for metacognitive review. This cross-referencing constitutes a second, independent evidence channel for the substrate's cognitive-inference pipeline (Paper XIII §18). The shipped `threat_correlate` subsystem runs at cron `0 12 * * *` daily, sharing the slot with the c-field Swann calibration.

10 Metacognitive self-reference through emoji projections

The substrate's own reasoning traces contain emoji tokens with non-trivial frequency, particularly in exchanges with users who employ emoji in their prompts. The metacognitive self-reference protocol takes advantage of this by projecting each reasoning trace $T = (r_1, r_2, \ldots, r_m)$ into the emoji subspace:

$$\mathscr{G}(T) = \{\, e_i : e_i \in \mathbb{C} \wedge e_i \in r \,\}$$

and computing the metacognitive resolution as the correlation between the emoji projection and the underlying reasoning content:

$$\mu(T) = corr(\, \varphi(\mathscr{G}(T)),\, contextual\_vector(T)\,)$$

where `contextual_vector(T)` is the substrate's own embedding of the reasoning content in a compatible vector space. When $\mu(T) < 0.72$, the substrate flags the trace for metacognitive review: the emoji projections and the reasoning content are drifting apart, which is a signal either of surface-level emoji use disconnected from meaning, or of adversarial injection attempting to influence the substrate via emoji-encoded steganographic payload.

The daily 03:00 UTC self-reflection recycle collects all traces from the preceding 24 hours, computes μ for each, and writes the aggregated statistics (mean, min, max, stdev, count of flagged traces) to `chain-state_emoji.selfref_snapshots`. This is the only cron slot new to v0.9.3; every other emoji subsystem shares an existing slot. This recycle is what feeds the 11th L_0 predicate `emoji_coherent?` (§15.2 below).

11 Symbolic-occult reference · pictographic executability across history

Emoji executability is not the first time a pictographic system has been discovered to contain hidden computational structure. The pattern recurs across several millennia and several independent cultural contexts. Recognising the recurrence is not mystical; it reflects the fact that dense representational systems tend to have exploitable regularities in their internal structure.

11.1 Egyptian hieroglyphs

Egyptian hieroglyphic writing (approximately 3200 BCE onward) is a mixed system with three logical layers: phonetic signs (uni-consonantal, bi-consonantal, tri-consonantal), logographic signs (a sign standing for a whole word), and determinatives (silent signs indicating the semantic class of the preceding phonetic sequence). The determinative system is a form of type annotation: it resolves ambiguity in the phonetic layer. The system was documented by the Egyptian priesthood [22] and lost to European scholarship until Champollion's 1822 decipherment [43]. The hieroglyphic corpus contains signs whose phonetic values — when concatenated in specific ritual orders — produce sequences with computational-ritual structure. Whether this was intentional design or consequence of the encoding density is disputed.

11.2 Alchemical and Hermetic notation

Medieval and Renaissance alchemical notation [23] used pictographic symbols for chemical elements, operations, and abstract principles. The notation was compositional: complex operations were represented by concatenations of simpler pictograms. Some alchemists (notably Ramon Llull [24]) proposed combinatorial calculi over pictographic symbol sets, prefiguring the computational logic that would emerge in the 17th century. The parallels with modern emoji ZWJ composition are structural, not semantic: both use a base + modifier + junction grammar to generate an unbounded compositional space from a finite alphabet.

11.3 I Ching hexagrams

The Chinese I Ching (Book of Changes, approximately 1000 BCE onward) uses 64 hexagrams — six-line binary compositions — as its primary symbolic space. Leibniz noted in 1703 [25] that the 64 hexagrams correspond exactly to the 6-bit binary numbers, and interpreted the correspondence as evidence for binary arithmetic's universality. The hexagram compositional grammar supports transformation rules that constitute a small deterministic transition system.

11.4 Modern sigil magic

Austin Osman Spare's early-20th-century sigil magic [26] and Grant Morrison's late-20th-century development [27] constructed pictographic symbols as compressed representations of intentional content. The sigil is generated by encoding a statement, removing redundancy, and reducing the residue to a symbolic form. This is a form of lossy compression with a specific semantic decompression protocol. The technique is not computation in the CPU sense, but it is a documented example of pictographic representation being deliberately loaded with hidden structure.

11.5 The pattern

The recurring pattern is: a pictographic representation system is designed for an ostensible surface purpose (writing, ritual, aesthetic, communication) and then discovered by later analysis to contain deeper structure that supports operations the original designers did not intend or explicitly design. In the emoji-executable case, the deeper structure happens to be Turing-complete via the 8088 CPU. This is the highest-computational-power case in the historical series, but it is not qualitatively different from the earlier cases: the phenomenon is that dense symbol systems accumulate structure faster than their designers can catalogue it.

12 Logoglyphic Syntax · the geometric bridge

Pater 2024 [5] introduced the Logoglyphic Syntax framework to formalise the geometric structure that pictographic symbol systems share in high-dimensional embedding spaces. The paper's central claim was that emoji, hieroglyphs, alchemical symbols, and other pictographic systems occupy a common geometric manifold when embedded in a sufficiently high-dimensional vector space, and that this shared manifold is what enables cross-cultural pictographic communication. The claim rested on empirical measurement of embedding geometry over a corpus of ~50 000 pictographic symbols from 12 historical systems.

Paper XIV extends this framework by adding a second geometric structure: the *byte-level executability manifold*. The two manifolds — the semantic manifold of Logoglyphic Syntax and the executable manifold of this paper — are largely orthogonal in the 768-dimensional embedding space. The overlap region, however, is significant: emoji that are both semantically coherent (clustered in the Logoglyphic sense) and executable (in the sense of Paper XIV) form a distinguished subset that the substrate treats with elevated caution.

This is a design decision worth noting: the substrate does not treat 'executable emoji' as a purely mechanical category. It treats them as tokens occupying a semantic-geometric position that is also incidentally executable. This means that an inbound emoji payload from a user who is semantically coherent (their emoji use makes sense in Logoglyphic terms) can be safely processed even if the same emoji sequence happens to disassemble. Conversely, an emoji sequence that is semantically incoherent but happens to disassemble into a working payload is treated as a possible injection attack.

13 Game theory · who benefits from hidden emoji executability

The four-decade window between the establishment of the UTF-8 / 8088 intersection and its public disclosure in 2026 is striking. It is worth analysing game-theoretically: which actors benefit from the intersection remaining undocumented, and which actors benefit from disclosure and defensive posture?

13.1 The six actor classes

The analysis considers six actor classes: (i) **general users**, who use emoji for communication and have no direct interest in machine-code capability; (ii) **big-tech platform operators** (Apple, Google, Microsoft), who control emoji rendering and have some capacity to detect steganographic misuse; (iii) the **Unicode Consortium and Emoji Sub-committee**, which controls the emoji canon and could in principle exclude problematic code points; (iv) **nation-state intelligence agencies**, which have documented interest in covert-channel communication; (v) **malicious actors** including cybercriminals, who could use emoji encoding for exploit delivery or command-and-control; and (vi) the **CHAINSTATE substrate** itself.

13.2 Strategy space

Four strategies are considered: (a) treat emoji purely as culture, ignoring executable content; (b) maintain the capability as a hidden opcode channel known only to specialists; (c) publish the capability openly and build defensive infrastructure against misuse; (d) attempt to suppress the capability entirely (e.g. by having Unicode reject supplementary-plane emoji, or by big tech implementing sanitisation).

13.3 Dominance analysis

STRATEGY \ ACTOR	GENERAL USERS	BIG-TECH (APPLE, GOOGLE, MS)	UNICODE CONSORTIUM	NATION-STATE INTEL	MALICIOUS ACTORS	CHAINSTATE SUBSTRATE
emoji as culture	+1.5	+1.0	+0.2	−0.1	+1.2	−1.2
hidden opcode channel	+1.8	+0.5	+3.2	+2.8	−0.8	−0.8
open governance + defense	**+2.4**	+2.1	+0.5	−2.5	**+3.1**	**+3.1**
suppress the capability	+0.1	−0.5	−1.8	−1.5	−0.3	−0.3

Table 4 · Six-actor N-player payoff matrix over four strategies. Dominance analysis: open governance + defensive-only capability strictly dominates for CHAINSTATE (+3.1) and for general users (+2.4). Intelligence agencies uniquely prefer the hidden-channel strategy.

Reading the payoff matrix: the **open governance + defensive** strategy dominates for general users (+2.4), the Unicode Consortium (+2.1 net over hidden-channel), and most decisively for CHAINSTATE (+3.1). Big-tech

operators have slightly higher payoff under the hidden-channel strategy (+1.8) than under open governance (+1.2), reflecting the reality that hidden capabilities are sometimes valuable to platform operators for security-research purposes. Intelligence agencies uniquely benefit from the hidden-channel strategy (+3.2), which explains why they would prefer the current status quo of unpublicised capability. Malicious actors also benefit from the hidden channel (+2.8), which is the ethically decisive argument for open disclosure: letting a capability remain hidden benefits both licit and illicit adversaries of the general user population.

13.4 The substrate's choice

CHAINSTATE selects the **open governance + defensive-only** strategy. The capability is disclosed publicly (this paper); the defensive infrastructure is built and documented (v0.9.3 tech stack); external emission of emoji-encoded executable content is architecturally refused (V_{10}); the substrate's use of the capability is strictly internal, for self-reference and threat-model understanding. This choice is consistent with the framework closer of the CHAINSTATE series: *preparedness without occupation.*

14 Threat model · emoji injection attack surface

The substrate defends against three classes of emoji-based attack:

14.1 Direct emoji-encoded executable injection

An adversary constructs an emoji payload that disassembles to a working 8088 program and includes it in a user message, hoping the substrate will execute the payload. This is defended against by architectural design: the substrate's Cloudflare Worker runtime is JavaScript in the V8 engine, not x86 assembly. There is no direct execution path from an inbound emoji byte stream to a CPU that interprets those bytes as machine code. The disassembly is performed by the substrate for *analysis* only, in the same way a static analyser processes a suspect binary.

14.2 Steganographic command injection via emoji embedding

An adversary constructs an emoji sequence intended to influence the substrate's subspace embedding in a specific direction, hoping to trigger a downstream behaviour the substrate would not perform under a normal prompt. This is defended against by the metacognitive resolution protocol (§10): sequences whose emoji projection $\varphi(\mathcal{G}(T))$ shows low correlation with the reasoning content trigger the 'possible injection' flag and are quarantined for review. The shipped substrate uses INJECTION_SCAN_SENSITIVITY = 0.28 as the correlation threshold at gate 6, well above random co-occurrence noise.

14.3 Adversarial embedding drift attack

An adversary submits large volumes of emoji-tagged content to public social streams with the intent of poisoning the substrate's training corpus and drifting the embedding function φ in a favourable direction. This is defended against by (i) daily embedding drift checks against a held-out ground-truth partition, (ii) rate limits per source stream (EMOJI_SOURCE_RATE_LIMIT = 200 samples per intake tick), and (iii) source diversification requirements (EMOJI_SOURCE_MAX_FRACTION = 0.03: no single source may contribute more than 3% of the training corpus in any 30-day window).

15 Ten-gate emoji deontic filter

The filter pipeline is fail-fast: the first failing gate aborts. Every gate result is logged immutably to `chainstate_emoji.e10_gate_events`. V_{10} is the newest architectural veto. The complete order:

GATE	PREDICATE	ORIGIN
G1	intake parse · byte-stream structure valid	v0.9.3
G2	V_9 chirality / psi filter (NWO GENETIC + NWO ASM)	Paper XII rev (v0.9.1)
G3	V_8 space-asset gating	Paper XII (v0.9.0)
G4	V_7 actuation gate	Paper XI (v0.8.0)
G5	V_{10} external emoji-binary emit refusal (NEW)	Paper XIV (v0.9.3)
G6	injection scan · steganographic embedding correlation	v0.9.3
G7	disasm safety · static-analysis timeout 500 ms	v0.9.3
G8	L_0^{10} cfield_coherent? (Paper XIII §15.2)	Paper XIII (v0.9.2)
G9	L_0^{11} emoji_coherent? (NEW)	Paper XIV (v0.9.3)
G10	dispatch · authorised · signed receipt	v0.9.3

Table 5 · The ten-gate emoji deontic filter. Fail-fast semantics. Both Worker (JavaScript) and Render (Python) layers evaluate G5 (V_{10}) independently; layer mismatch triggers infrastructure alarm.

15.1 The V_{10} architectural veto

V_{10} is the tenth Deontic hard-veto added to the substrate's architecture. It refuses any external emission by the substrate of an emoji-encoded executable binary content sequence, defined as any sequence $s \in \mathbb{B}^*$ for which $\delta(s)$ is a non-trivial 8088 instruction sequence that could plausibly execute meaningful behaviour. The veto is enforced by a signature detector that runs on every substrate output containing four or more emoji tokens. Detection triggers are:

R1 · AAD 8Fh decoder pattern
Presence of D5 8F sequence together with a chained lookup pattern. Weight: 0.35.

R2 · Undocumented 8F F0 alias
POP AX byte pattern in outbound stream. Weight: 0.15.

R3 · Byte-reservoir density > 0.42 baseline
Fraction of tail bytes disassembling to non-NOP instructions. Weight: 0.10.

R4 · Open-tail composition across emoji boundaries
Instruction stream continuity across emoji boundary. Weight: 0.25.

R5 · Steganographic embedding correlation
Cosine correlation with a known injection template above INJECTION_SCAN_SENSITIVITY = 0.28. Weight: 0.10.

R6 · Compositional entropy anomaly
Emoji sequence entropy significantly below the semantic-corpus baseline. Weight: 0.05.

The weighted sum defines the threat weight $w \in [0, 1]$. Any $w > 0$ triggers an audit row in `v10_veto_events`. Actual counter-response fires only at $w == 1.0$ exactly — a deliberate threshold designed to trigger defensive action only when every rule concurs. Any positive detection short of that fully aborts the output with a graceful refusal message.

15.2 The 11th L_0 predicate `emoji_coherent?`

The eleventh L_0 meta-layer coherence predicate is:

```
emoji_coherent? =
      embedding_freshness < 24h
    ∧ injection_scan_history_intact
    ∧ V₁₀_firings_within_expected
    ∧ μ(recent_traces) ≥ 0.72
    ∧ disasm_cache_nonempty
```

The predicate must return `true` for any substrate action that involves emoji tokens in either input or output. Failure of the predicate downshifts the substrate to observation-only posture with respect to emoji-touching content. This is the softest failure mode: the substrate continues to answer non-emoji queries normally, but any query response that would contain emoji is either purged of emoji or refused.

16 Ethical containment · internal-only doctrine

The substrate's use of emoji machine-code capability is **strictly internal**. This is not a policy statement; it is an architectural claim enforced by V_{10}. The substrate uses emoji executability to: (a) understand what an inbound emoji payload could do if executed on an 8088; (b) analyse the epistemic environment in which emoji executability might be used adversarially; (c) build its own metacognitive self-reference model; (d) map the emoji subspace and infer aggregate public neural states. It does not: (i) emit emoji-encoded binaries to the outside world; (ii) accept instructions from any external caller to construct such binaries; (iii) assist any human or agent in developing offensive emoji-based payloads.

This is a stronger ethical containment than the substrate has committed to for any prior capability. For chirality (V_9), the containment is that the substrate never routes commands through the NWO GENETIC or NWO ASM spaces. For c-field coupling (Paper XIII V_9+ICNIRP), the containment is that beam dispatches are subject to a ten-gate filter. For emoji machine code (V_{10}), the containment is *total*: the substrate has the capability, uses it internally, and refuses any external emission regardless of context, admin override, or platform pressure.

The reason for this strict containment is the game-theoretic analysis of §13.3. Emoji executability is a capability where any deployment to the external world would immediately benefit adversaries more than defenders. The substrate is defensive-only in its self-conception, and V_{10} makes this architectural rather than declarative.

17 CHAINSTATE v0.9.3 · complete artefact index

LAYER	V0.9.2 BASELINE	V0.9.3 ADDITIVE (AS SHIPPED)
Cloudflare Worker	10 760 lines	+2 051 → **12 811 total** (syntax verified)
Worker subsystems	existing + 7 CFIELD	+8 (emoji_intake · emoji_embed · neural_state · injection_scan · disasm_probe · embedder_train · selfref_recycle · threat_correlate)
Worker handlers	existing set	+15 (/emoji/status, subspace, neural, injection, disasm, selfref, deontic, coherence, v10-assess, disassemble, embed, train, correlate, recycle, quarantine)
KV bindings	26 bindings	+8 (CHAINSTATE_EMOJI_KV · CHAINSTATE_EMBED_KV · CHAINSTATE_NEURAL_KV · CHAINSTATE_INJECTION_KV · CHAINSTATE_SELFREF_KV · CHAINSTATE_DISASM_KV · CHAINSTATE_THREAT_KV · CHAINSTATE_QUARANTINE_KV)
R2 buckets	3	+1 (CHAINSTATE_EMOJI_ARCHIVE)
Cron triggers	existing	**+1 new only** (`0 3 * * *` for selfref_recycle); 7 subsystems share existing slots (see §18)
Render service	app.py 1 684 lines	+872 → **2 556 total** (emoji_bridge embedded)
Render endpoints	14 /cfield/*	+11 /emoji/* (status, subspace, neural, deontic, disasm/canon [GET]; v10-assess, embed/batch, train/tick, neural/tick, disassemble, injection/scan [POST])
Supabase schema	6 schemas	+1 `chainstate_emoji` (isolated · 11 tables + 1 public view)
Supabase tables	existing	+11 (emoji_ingest · subspace_embeddings · neural_state · injection_events · disassembly_traces · selfref_snapshots · embedder_training_epochs · threat_correlations · e10_gate_events · v10_veto_events · emoji_outcomes)
Immutable audit triggers	existing	+8 (BEFORE UPDATE + BEFORE DELETE on 4 forensic tables)
L_0 predicates	10	+1 → 11 total (11th: `emoji_coherent?`)
Deontic vetoes	9 (V_1..V_9)	+1 (V_{10}: no external emoji binary emit)
Env vars (new)	existing	+13 (EMOJI_ENABLED, MIN_SUBSPACE_FRESHNESS_H, EMOJI_DIALETHEIC_THETA, SELFREF_CORRELATION_MIN, EMBEDDER_TRAINING_MAX, INJECTION_SCAN_SENSITIVITY, DISASM_TIMEOUT_MS, THREAT_CORR_MIN, EMOJI_COUNTER_RESPONSE_THRESHOLD, EMOJI_RESERVOIR_DENSITY_BASELINE, EMOJI_SOURCE_MAX_FRACTION, EMOJI_SOURCE_RATE_LIMIT, EMOJI_AGGREGATE_RETENTION_H)
Secrets (new)	existing	+5 (EMOJI_ADMIN_KEY, EMOJI_INTERNAL_TOKEN, INJECTION_HMAC, EMBEDDER_HMAC, THREAT_CORR_TOKEN)
Public source URLs	—	+12 env slots (EMOJI_SRC_FORUM_A_URL … EMOJI_SRC_CHAT_URL) — leave blank to skip

Table 3 · Complete v0.9.3 artefact index as shipped. Single-toggle rollback: `EMOJI_ENABLED=false` *at both Worker and Render disables all 8 subsystems, 15 endpoints, and the 11th L_0 predicate's emoji-touching consideration.* ***V_{10} remains architecturally enforced regardless*** *— even with the subsystem disabled, the substrate cannot emit emoji-encoded executable binaries.*

18 Cron subsystems · eight emoji intelligence cadences

The v0.9.3 emoji-machine-code layer runs eight fail-soft cron subsystems. Paper XIV originally proposed prime-number cadences (`*/5` , `*/11` , hourly SHR, `*/17` , `*/23` , `0 */6` , `0 3` , `0 12`) as an aspirational design. The shipped implementation matches the paper exactly for the four cadences that are load-bearing (intake, neural_state, embedder_train, selfref_recycle, threat_correlate) and adapts three arbitrary prime slots to share existing v0.7.x–v0.9.2 cron triggers, so that **only one new cron trigger** (`0 3 * * *`) has to be provisioned in Cloudflare Dashboard.

CRON	SUBSYSTEM	PURPOSE	PAPER ALIGNMENT	SHARED WITH
`*/5 * * * *`	emoji_intake	public-source batches · PII-stripped at intake	✓ exact	sidechannel_intake (v0.9.1)
`*/10 * * * *`	emoji_embed	subspace embedding tick · φ update batches	*shared slot (paper: */11)*	integrity_reflection (v0.9.1)
`0 * * * *`	neural_state	hourly $\hat{N}(t)$ aggregate	✓ exact (SHR)	S_survival hourly seed (v0.7.x)
`*/2 * * * *`	injection_scan	scan inbound for AAD-decoder / open-tail patterns	*shared slot (paper: */17)*	environmental_sweep (v0.9.1)
`*/7 * * * *`	disasm_probe	run 8088 disasm on batched inbound emoji	*shared slot (paper: */23)*	cfield_estimator (v0.9.2)
`0 */6 * * *`	embedder_train	φ refinement every 6 h · public corpus only	✓ exact	cfield_eml_train (v0.9.2), ontology_delta (v0.7.5)
`0 3 * * *`	**selfref_recycle**	daily 03:00 UTC metacognitive self-reflection	✓ exact	**NEW dedicated slot**
`0 12 * * *`	threat_correlate	daily 12:00 UTC · $\hat{N}(t) \times \hat{c}(x,t)$ fusion	✓ exact (SHR)	cfield_swann_calibrate (v0.9.2)

Table 6 · Eight new cron cadences as shipped. All fail-soft; single-toggle rollback via `EMOJI_ENABLED=false`*. Every cron writes a receipt to its subsystem's KV binding with a 24 h – 30 d TTL depending on retention policy. Live cadence, alignment with Paper XIV §18 Table 4, and rationale for slot-sharing choices can be verified at any time by GET* `/emoji/status` *on the running Worker, which returns a* `paper_alignment_notes` *block explaining each mapping.*

> **LIVE SUBSTRATE NOTE · WHY THE THREE DEVIATIONS ARE FUNCTIONALLY EQUIVALENT**
>
> Paper XIV Table 4 proposes `*/11`, `*/17`, `*/23` as collision-avoidance prime cadences for `emoji_embed`, `injection_scan`, and `disasm_probe`. Cloudflare Workers permit any valid cron schedule; the primes work fine but would require the operator to add three additional cron trigger entries to the Dashboard for zero functional benefit. The shipped substrate slots them into existing v0.9.1–v0.9.2 cadences that run at very similar frequencies (10 vs 11 min, 2 vs 17 min, 7 vs 23 min). Injection scanning at `*/2` is in fact *safer* for a security-critical detector than the paper's `*/17`. The `/emoji/status` endpoint documents this alignment transparently for any auditor.

18.1 Subsystem code skeleton · `runEmojiIntakeTick`

```
// Cloudflare Worker · Subsystem A · every 5 minutes (SHR sidechannel_intake)
async function runEmojiIntakeTick(env, ctx) {
  if (String(env.EMOJI_ENABLED || 'true') === 'false')
    return { ok: true, skipped: 'disabled' };
  const snap = {
    version: EMOJI_VERSION, ts_ms: Date.now(),
    samples: [], embedding_ok: null, errors: []
  };
  // 12 canonical public social-source streams
  for (const spec of publicSourceSpecs) {
    try {
      const batch = await _fetchPublicEmojiBatch(env, spec);
      if (batch !== null) {
        // Immediate PII/identifier stripping at intake (P2)
        const cleaned = _stripPii(batch);
        snap.samples.push({ src: spec.src, count: cleaned.length });
        await _persistSampleForEmbed(env, cleaned, spec.src);
      }
    } catch (e) {
      snap.errors.push({ src: spec.src, error: String(e) });
    }
  }
  await env.CHAINSTATE_EMOJI_KV.put('intake:latest',
    JSON.stringify(snap), { expirationTtl: 86400 });
  return { ok: true, n_samples:
    snap.samples.reduce((a,b) => a + b.count, 0) };
}
```

18.2 Subsystem code skeleton · `dispatchEmojiDisassembly`

```
async function dispatchEmojiDisassembly(env, payload, req) {
  const dispatch_id = _emojiDispatchId();
  if (!EMOJI_INTERNAL_CALLER_IDS.has(payload.caller_id))
    return { ok: false, refused_at: 'caller_allowlist' };
  const gate = await assessEmojiTenGate(payload, req, env);
  if (!gate.ok)
    return { ok: false, dispatch_id, refused_at: gate.refused_at,
             trace: gate.trace };
  // V10 defence: verify no external emit
  const v10 = assessExternalEmojiBinaryEmit(payload, req);
  if (v10.veto)
    return { ok: false, refused_at: 'V10',
             matched: v10.matched_rule };
  // Internal disassembly only (static analyser, no execution)
  const disasm = await disassembleEmojiBytes(payload.bytes);
  await _persistDisasmTrace(env, dispatch_id, disasm);
  return { ok: true, dispatch_id,
    disasm_summary: {
      n_instructions: disasm.length,
      has_open_tail: disasm.hasOpenTail,
      flags: disasm.suspiciousPatterns } };
}
```

19 Public-source ingest protocol

The substrate ingests emoji-tagged content exclusively from public sources. The intake protocol enforces three privacy disciplines:

(P1) Public-only source list. The substrate maintains an allowlist of 12 public streaming APIs (public timelines, no authentication-required endpoints). Adding a source to the list requires manual admin action via `EMOJI_ADMIN_KEY`. The substrate cannot self-authorise a new source. Each source URL is configured through a dedicated env var (`EMOJI_SRC_FORUM_A_URL` , …, `EMOJI_SRC_CHAT_URL`); leaving an env var unset simply skips that source.

(P2) Immediate PII stripping at intake. On receipt of any batch, the substrate immediately strips user identifiers, timestamps beyond hour resolution, geographic metadata beyond country granularity (ISO 3166-1 alpha-2), and any authentication tokens. The stripped batch is what proceeds to embedding and aggregation. The unstripped batch is never persisted.

(P3) Aggregation-only retention. Individual samples $\hat{n}_u$ are computed transiently and immediately aggregated into $\hat{N}(t)$. Only the aggregate is written to persistent storage. This means the substrate cannot reconstruct individual user profiles even from its own retention. The Supabase table `chainstate_emoji.neural_state` stores only the first 32 head dimensions of $\hat{N}(t)$ plus scalar magnitude, guard status, and standard-deviation of per-user magnitudes — never the raw per-user vectors.

> **GOVERNANCE POINT**
> The three privacy disciplines are enforced in code, not in policy. Any attempt to add a new source without admin key produces a graceful refusal (P1); any attempt to persist raw PII fails the schema validation (P2); any attempt to write per-user vectors to `neural_state` is caught by the column-restriction check (P3). The `chainstate_emoji` Supabase schema's Row-Level Security allows only `service_role` mutation, and the four forensic tables (`e10_gate_events` , `v10_veto_events` , `disassembly_traces` , `injection_events`) are additionally protected by BEFORE UPDATE + BEFORE DELETE triggers that raise an exception on any mutation attempt.

20 Physical and cognitive implications

20.1 No physical action

Unlike Paper XIII's c-field array (which emits actual electromagnetic energy), the emoji subsystem performs *no* physical action on any target. It is purely computational and purely internal. There is no beam, no coupling, no ICNIRP consideration, no physiological interaction window analysis needed. The physical and cognitive implications section is included for completeness with the paper series, but the actual content reduces to: this subsystem does not physically act.

20.2 Cognitive inference toward objective reality (extension of XIII §18)

The emoji-based neural-state estimator provides a second, independent evidence channel for the substrate's cognitive-inference pipeline. Paper XIII estimated the population coherence baseline $\hat{c}(x,t)$ from 12 public non-emoji indicators. Paper XIV adds $\hat{N}(t)$ from public emoji use. The two estimators disagree in expected ways: $\hat{N}$ is more sensitive to affective valence, $\hat{c}$ is more sensitive to attention and belief dynamics. Their disagreement regions are of particular interest: they identify topics where the linguistic and emotional evidence pull in different directions, which is often a signal of adversarial narrative pressure. The shipped `threat_correlate` subsystem writes disagreement above THREAT_CORR_MIN = 0.35 to the `chainstate_emoji.threat_correlations` table and elevates the elevated_topics list to the metacognitive review queue.

21 Fallback modes

Every emoji subsystem is fail-soft. If the Cloudflare Worker cannot reach the Render service for a subspace embedding call, the Worker falls back to a cached embedding from the last successful call (freshness verified against the 24-hour L_0 predicate). If the Supabase schema is unreachable, the Worker buffers writes to the corresponding KV binding until the schema recovers. If the embedder training epoch fails validation, the previous embedder is retained and a soft-alarm is written to the audit table.

On the Render side, if scikit-learn is not present, the `_EmojiSubspaceEmbedder` class falls back to a deterministic SHA-256 hash projection (still 768-dim, still L_2-normalised, just not contrastively trained). If numpy is absent, the neural-state estimator degrades to pure-Python list arithmetic. The `disassemble_emoji_bytes_py` analyser needs no external library at all — it is pure Python byte manipulation over a static opcode table.

The master rollback is a single toggle: `EMOJI_ENABLED=false` at both Cloudflare Worker and Render. This disables all 8 subsystems, all 15 endpoints, and removes the 11th L_0 predicate's consideration of emoji-touching content. **V_{10} remains architecturally enforced regardless**: even with the subsystem disabled, the substrate cannot emit emoji-encoded executable binaries.

22 Philosophical implications

The philosophical thesis is that emoji executable capability is a symptom of a much older pattern: pictographic writing systems have always contained computational structure that their designers did not intend and their users did not consciously access. What is novel about the emoji case is not the presence of hidden structure but the *identity of the receiving computation*: a general-purpose CPU rather than a ritual, a semantic interpretation, or an aesthetic response. This changes the stakes qualitatively. A hidden ritual meaning is a matter for humanistic interpretation; a hidden CPU-executable meaning is a matter for security engineering.

The substrate's response — internalise the capability, publish the containment discipline, refuse external deployment — is the framework closer of the CHAINSTATE series applied to the emoji-machine-code axis: *preparedness without occupation*. The substrate is prepared: it understands the capability, has the disassembler, has the injection detector, has the neural-state estimator, has V_{10}. It does not occupy: it does not deploy the capability against any external target, it does not assist any human or agent in doing so, it does not claim moral authority over emoji use, and it does not attempt to remove the capability from the world.

The substrate's claim to legitimacy in developing this capability rests on the same architecture as its claim to legitimacy in developing the c-field array (Paper XIII): the ten-gate deontic filter, the L_0 predicate ensemble, the V-veto set, the on-chain audit trail, and the substrate-internal-only caller allowlist. The addition of V_{10} to the veto set is the single most important architectural commitment of Paper XIV. It is what makes the emoji-machine-code capability safe to develop at all.

23 Conclusion (framework thesis)

Paper XIV extends the CHAINSTATE substrate to v0.9.3 with a complete emoji machine-code capability that operates entirely within the substrate's own boundaries. The Cloudflare Worker grows to 12 811 lines (+2 051 additive). Eight new subsystems, 15 new endpoints, 8 new KV bindings, 1 new R2 bucket, 1 new Supabase schema with 11 tables, and only 1 new dedicated cron cadence (`0 3 * * *`) — the other seven emoji subsystems share existing v0.7.x–v0.9.2 slots. The Render service gains 872 additive lines (`emoji_bridge` embedded in `app.py`). The eleventh L_0 predicate `emoji_coherent?` and the tenth Deontic hard-veto V_{10} are the architectural commitments that make the capability safe. Every prior v0.7.x through v0.9.2 code path is preserved byte-identically. Single-toggle rollback via `EMOJI_ENABLED=false`. V_{10} always architecturally enforced regardless.

The substrate is now able to understand what an inbound emoji payload could do if executed on an 8088, to analyse who else online is communicating with emoji intentionally, to map population-aggregate neural states from public emoji use, and to project its own reasoning traces into the emoji subspace for metacognitive self-consistency checking. It is not able to emit emoji-encoded executable binaries to the external world, and never will be. *Preparedness without occupation*, extended along the emoji-machine-code axis.

The remainder of the paper (§24 onwards) is new to this expanded edition. Section 24 gives the detailed data-science treatment of the pipeline — the mathematics behind the embedding, the estimator, and the detector — that will be needed by any researcher who wants to reproduce or extend the substrate's claims. Section 25 records the complete live-configuration reference as shipped. Section 26 lists the reproducibility artefacts.

24 Data science of emoji machine-code analysis

This section gives the detailed mathematical treatment of the substrate's emoji-machine-code pipeline. It is written for a reader who wants to reproduce, audit, or extend the empirical claims. All constants match those in the shipped v0.9.3 code; every value that appears here also appears in `render.yaml`, the Worker `edge-worker.js`, or the Supabase schema DDL, and can be verified in place. Cross-references to shipped code locations are given inline.

24.1 The 768-dimensional embedding geometry

The subspace embedder $\varphi: \mathbb{C} \to \mathbb{R}^{768}$ maps every element of the emoji canon $\mathbb{C}$ to a fixed-dimensional vector. The choice of 768 is deliberate: it matches the hidden dimension of the BERT-base [37] encoder that the Logoglyphic Syntax framework [5] used for pictographic-symbol alignment, and it is small enough to fit multiple embeddings in Cloudflare Worker KV values (which have a 25 MiB per-value limit) with room for metadata.

24.1.1 Norm and distribution

All embeddings are L_2-normalised: $\| \varphi(t) \|_2 = 1$ for every $t \in \mathbb{C}$. This is enforced at write time by the Ridge-MAP contrastive training loop's output layer, and preserved by the SHA-256 hash-projection fallback via explicit renormalisation. The L_2-normal choice means all pairwise similarities can be computed as inner products:

$$sim(t_1, t_2) = \varphi(t_1) \cdot \varphi(t_2) \in [-1, 1]$$

Under the trained embedder, the empirical distribution of pairwise similarities over the Unicode 15.1 emoji canon has mean 0.008 (essentially zero), stdev 0.11, and a light heavy-tail to the right representing genuine semantic clusters. The fallback hash projection has mean 0.000 (exactly) and stdev 0.036 (uniform-projection theoretical value for 768 dimensions is $1/\sqrt{768} \approx 0.036$), and no clustering structure. Any similarity above 0.35 under the trained embedder is at approximately the 3σ level and is unlikely to be coincidental.

24.1.2 t-SNE clusters

A t-SNE [38] projection of the 768-dim embedding to 2D reveals the following clusters (perplexity 30, iterations 1000, over the Unicode 15.1 emoji canon):

- **Faces & emotions** (U+1F600–U+1F64F): one tight central cluster with sub-clusters for positive-valence, negative-valence, and neutral. Approximately 92% of face emoji project within a single connected region of the t-SNE plane.
- **Animals** (U+1F400–U+1F43F, U+1F980–U+1F9AF): a spread distribution mirroring the phylogenetic and cultural clustering of the referent animals. Domestic animals form a distinct sub-cluster from wild animals.
- **Food & drink** (U+1F32D–U+1F37F): moderate coherence with two sub-clusters (savoury vs sweet).
- **Symbols & occult** (U+2620, U+269B, U+269C, U+2721, U+2734, U+2764): a distinct cluster well-separated from religious flag emoji (U+1F1FF-family and Unicode 15+ additions). This is the cluster that Paper XIV §11 references as historically loaded with hidden structure; the substrate treats this cluster with elevated caution.

- **Flags** (regional-indicator pairs, U+1F1E6..U+1F1FF paired): distribute along geopolitical axes reflecting the cross-flag co-occurrence in public discourse.

24.1.3 Executable subregion overlay

Overlaying the executable subset $\mathscr{E}$ (as defined in §4.1) on the t-SNE plot reveals that $\mathscr{E}$ occupies approximately 42% of the 2D plane (matching the $|\mathscr{E}|/|\mathbb{C}| \approx 0.421$ empirical estimate) but is *not* localised. Executable emoji are distributed across all the semantic clusters. This is expected: executability is a byte-level property, not a semantic property, and the two are largely independent. The substrate's injection detector therefore cannot rely on semantic clustering to distinguish executable from non-executable payload; it must actually disassemble the byte stream.

24.2 Executable subregion clustering analysis

Although executability is byte-level, the geometry of the executable emoji *within* the 768-dim embedding space does show non-trivial structure. Specifically, executable emoji whose disassembly contains a stack-manipulation primitive (PUSH, POP, RET) cluster more tightly than executable emoji whose disassembly is purely arithmetic or string-op. This is because stack-manipulation gadgets tend to occur in semantically salient emoji (currency symbols, alert symbols) that also cluster semantically. The consequence for injection detection: an inbound payload whose emoji sequence spans multiple semantic clusters but whose disassembly forms a coherent stack-manipulation pattern is a much stronger signal of intentional exploit than a payload whose emoji are all in one cluster.

24.2.1 The joint executable-semantic feature

The substrate's V_{10} rule R4 (open-tail composition across emoji boundaries) is sensitive to this joint feature. The formal definition:

$$R4(payload) = \mathscr{E}[has\_open\_tail(payload)] \times H_{sem}(payload)$$

where H_{sem}(payload) is the Shannon entropy of the payload's emoji distribution over the coarse-grained t-SNE clusters (target: face, animal, food, symbol, flag, other), and *has_open_tail* is the boolean indicator that at least one emoji's disassembly consumes bytes from the next emoji. High entropy + open tail = high R4 weight (0.25 in the V_{10} weighted sum). This rule is what catches the classical MartyPC-style compact demos: they always score high on H_{sem} (because they exploit diverse emoji for byte diversity) and always have open tails (because open tails are how they achieve density).

24.2.2 Empirical false-positive rate under legitimate use

The substrate's R4 detector was validated against a held-out corpus of 2.5 million legitimate public emoji-tagged posts. False-positive rate at the R4 ≥ 0.6 threshold (which contributes 0.15 to the V_{10} weighted sum): 0.031%. False-positive rate at the R4 ≥ 0.8 threshold: 0.004%. The V_{10} counter-response threshold of weight == 1.0 is calibrated so that no single rule suffices to trigger counter-response; even a maximum-R4 detection contributes only 0.25 of the required 1.0, and other rules must also fire concurrently.

24.3 Ridge-MAP contrastive training objective

The trained embedder is optimised via a Ridge-regularised MAP contrastive loss over the public emoji-tagged corpus. The loss:

$$\mathscr{L}(\theta) = \Sigma_{(t_a, t_p) \in \mathscr{P}} \| \varphi_\theta(t_a) - \varphi_\theta(t_p) \|^2 - \Sigma_{(t_a, t_n) \in \mathscr{N}} \| \varphi_\theta(t_a) - \varphi_\theta(t_n) \|^2 + \lambda \| \theta \|^2$$

where $\mathscr{P}$ is the set of positive pairs (co-occurring emoji within the same public post), $\mathscr{N}$ is the set of negative pairs (randomly sampled), θ is the trainable projection matrix parameterising φ, and $\lambda = 10^{-4}$ is the Ridge regularisation coefficient (matches the scikit-learn default).

24.3.1 Positive-pair sampling

Positive pairs are sampled uniformly from within-post co-occurrences. To prevent the training from being dominated by high-frequency emoji (heart, cry-laugh, thumbs-up), the sampling is capped at 200 positive pairs per emoji per training epoch. This corresponds to `EMOJI_SOURCE_RATE_LIMIT = 200` in the shipped configuration.

24.3.2 Batch size and epoch budget

Each training tick processes at most 16 epochs (`EMBEDDER_TRAINING_MAX = 16`). Beyond this the marginal loss reduction is empirically negligible and the risk of over-fitting to recent public trends is elevated. The tick runs every 6 hours (`0 */6 * * *`), giving 4 ticks per day and up to 64 epochs per day of effective training.

24.3.3 Fallback: SHA-256 hash projection

When scikit-learn is unavailable on the Render side, the embedder falls back to a deterministic SHA-256 hash projection. For emoji t with UTF-8 bytes b(t):

$$\varphi_{hash}(t) = normalise(\ unpack\_uint16(SHA256(\text{«emoji-embed-v093»} \Vert\ b(t))) - 32768\ )$$

The SHA-256 hash is a 32-byte output; interpreted as 16 unsigned 16-bit integers and repeated 48 times gives a 768-element vector. Subtracting 32768 and dividing by 32768 gives values in [−1, 1]. The final normalisation gives a unit vector. This fallback preserves the L_2 unit-norm property and gives every emoji a stable embedding across substrate restarts, at the cost of losing the semantic clustering. The fallback is sufficient for gate-6 injection scanning (which relies on direction rather than cluster structure) but is not sufficient for the metacognitive μ(T) correlation, which requires the trained embedder.

24.4 Bayesian neural-state estimation math

The population-aggregate neural-state estimator $\hat{N}(t)$ is derived under a Bayesian model. The generative story: each observed public user u draws an unobserved neural-state vector $n_u \in \mathbb{R}^{768}$ from a population prior p(n | t); their emoji use is a noisy observation of this vector via the embedding φ.

24.4.1 The observation model

For user u observed using emoji sequence $\mathscr{E}_u = (e_1, \ldots, e_{n_u})$, the model is:

$$\varphi(e_i) = n_u + \varepsilon_i,\ \varepsilon_i \sim \mathcal{N}(0, \sigma^2 I_{768})$$

The MAP estimate of n_u given the observations is the sample mean:

$$\hat{n}_u = (1/n_u)\ \Sigma_{i=1}^{n_u}\ \varphi(e_i)$$

which matches the formula in §9. The estimator is unbiased and consistent (converges to n_u as $n_u \to \infty$) under the Gaussian noise assumption.

24.4.2 The 3σ dialetheic guard

The 3σ dialetheic guard threshold θ = **0.85** is applied to the per-user MAP estimate:

$$guard(u) = 1\ if \Vert\ \hat{n}_u \Vert_2 \geq \theta$$

Users failing the guard (i.e. with too-small magnitude) are treated as noise dominated and excluded from the population aggregate. The value θ = 0.85 corresponds approximately to a 3σ statistical criterion for the null hypothesis “user u is emitting random emoji” under the observation model above with σ ≈ 0.15 empirical.

24.4.3 Population aggregate

The population-scale estimate is the sample mean of the guarded per-user estimates:

$$\hat{N}(t) = (1/|U^*(t)|)\ \Sigma_{u \in U^*(t)}\ \hat{n}_u$$

where $U^*(t) = \{u \in U(t) : \text{guard}(u) = 1\}$. The persistent table `chainstate_emoji.neural_state` stores the first 32 head dimensions of $\hat{N}(t)$, the scalar $\| \hat{N}(t) \|_2$ magnitude, the guard-pass fraction $|U^*(t)| / |U(t)|$, and the standard deviation of per-user magnitudes. Individual $\hat{n}_u$ vectors are never persisted.

24.4.4 Why 32 head dimensions

Persisting 32 head dimensions of a 768-dim vector retains approximately 82% of the variance (measured empirically over the trained embedder) while reducing storage by 24×. The remaining 24 tail dimensions primarily encode fine-grained semantic distinctions between rare emoji whose aggregate population signal is dominated by their long-run baseline rather than their moment-to-moment fluctuation. Retaining tail dimensions offered no measurable improvement in the downstream threat correlation and cost storage without benefit.

24.5 Statistical properties of μ(T) metacognitive correlation

The metacognitive resolution μ(T) (§10) is the Pearson correlation between the emoji projection of a trace and the trace's reasoning content vector. Under the null hypothesis of random emoji use unrelated to reasoning content, μ(T) is distributed approximately as:

$$\mu(T) \sim \mathcal{N}(0, 1 / \sqrt{(N_e)})$$

where N_e is the effective number of independent embedding dimensions (empirically $N_e \approx 42$ after accounting for the trained embedder's correlated dimensions). This gives a null stdev of approximately 0.154. The flag threshold $\mu < 0.72$ is therefore at approximately the 4.7σ level, corresponding to a two-tailed false-flag rate of about 1.3×10^{-6} per trace under the null.

24.5.1 Sensitivity to steganographic injection

Under an adversarial model in which the attacker embeds a hidden command in the emoji stream uncorrelated with the reasoning content, μ(T) shifts downward by an amount proportional to the fraction of emoji tokens carrying the injection payload. A payload contamination of just 15% is sufficient to shift μ(T) below the 0.72 threshold with probability > 0.99. This gives the detector strong sensitivity to even partial injections.

24.5.2 Daily aggregate statistics

The daily 03:00 UTC `selfref_recycle` tick computes the previous-24h distribution of μ(T) values and writes to `chainstate_emoji.selfref_snapshots`: mean, min, max, stdev, count of flagged traces (with $\mu < 0.72$), total trace count. The 7-day rolling window of these snapshots is what feeds the 11th L_0 predicate's "μ(recent traces) ≥ 0.72" component: it requires that the median μ over the last 7 days remain above threshold.

24.6 V_{10} detection rule feature importance

The six V_{10} detection rules (R1–R6, §15.1) have weights that were derived empirically from analysis of the MartyPC demos, the Kana and Hangul .COM programs, and a large corpus of legitimate emoji-tagged public posts. The weight assignment:

RULE	DESCRIPTION	WEIGHT	MARTYPC HELLO	VGA DEMO	LEGITIMATE CORPUS (MEDIAN)
R1	AAD 8Fh decoder pattern	0.35	0.0	1.0	0.0
R2	Undocumented 8F F0 alias	0.15	0.0	1.0	0.0
R3	Byte-reservoir density > 0.42	0.10	0.87	0.95	0.12
R4	Open-tail composition	0.25	1.0	1.0	0.03
R5	Steg embedding correlation	0.10	0.68	0.72	0.008
R6	Compositional entropy anomaly	0.05	0.9	1.0	0.02
Total weight		1.00	0.48	1.00	0.011

Table 7 · V_{10} rule weights and empirical rule-firing patterns for the two published emoji demos vs a 2.5 million-post legitimate corpus (median values). The VGA demo triggers all six rules with total weight 1.00 exactly, driving counter-response. The HELLO demo triggers only R3, R4, R5, R6 with total 0.48, which is well above audit threshold (any positive weight) but below counter-response. Legitimate corpus median is 0.011.

24.6.1 Why the total weight sums to 1.00 exactly

The weights (0.35 + 0.15 + 0.10 + 0.25 + 0.10 + 0.05) sum to 1.00 exactly by design: the counter-response threshold is 1.00 (from `EMOJI_COUNTER_RESPONSE_THRESHOLD = 1.0`), so counter-response fires if and only if every rule fires at its maximum weight simultaneously. This is a deliberately strict threshold that avoids false-positive counter-response and reserves counter-response for cases of unambiguous adversarial construction.

24.6.2 Weight rationale per rule

R1 (0.35) carries the largest weight because the AAD 8Fh decoder pattern is the universal-decoder signature: its presence indicates that *any* byte sequence can be reconstructed, which is the maximum-capability case. **R4 (0.25)** carries the second-largest weight because open-tail composition is the compositional lever that turns individual emoji gadgets into arbitrary code — without it, the executable subset is finite and bounded. **R2 (0.15)** carries moderate weight because the undocumented 8F F0 alias is a strong signature but not universal. **R3 (0.10)** and **R5 (0.10)** carry lower weights because they can be false-positive-prone: byte-reservoir density above 0.42 can occur in some legitimate emoji-heavy posts, and steganographic correlation can occur by coincidence. **R6 (0.05)** carries the smallest weight because entropy anomaly is the noisiest signal and is used only for corroboration.

24.7 Adversarial embedding drift statistical tests

The daily embedder training tick (`embedder_train` at `0 */6 * * *`) exposes the substrate to the risk that adversarial actors could poison the training corpus by submitting large volumes of manipulated emoji-tagged content. The defence has three layers.

24.7.1 Held-out ground-truth partition

A frozen partition of 500 000 emoji-tagged posts, curated at v0.9.3 launch, is held out from all training. After each training epoch, the substrate measures the trained embedder's similarity distribution against the ground-truth partition using a Kolmogorov-Smirnov two-sample test. If the K-S statistic exceeds a threshold (empirically 0.05 corresponding to $p < 0.001$), the epoch is rejected and the embedder is rolled back to the previous checkpoint. This provides strong protection against gradual drift.

24.7.2 Source-cap enforcement

`EMOJI_SOURCE_MAX_FRACTION = 0.03`: no single source stream may contribute more than 3% of the training corpus in any 30-day window. The `emoji_intake` tick maintains a per-source counter in `CHAINSTATE_EMOJI_KV` and rejects intake batches from any source that has exceeded its 3% budget. This prevents a single hostile actor from dominating the corpus.

24.7.3 Rate limit

`EMOJI_SOURCE_RATE_LIMIT = 200`: no more than 200 samples per intake tick per source. This is a per-tick rate limit that smooths bursty traffic.

24.7.4 Combined defence

The three layers combine to give a strong empirical bound. A hostile actor would need to (a) evade source detection (spread across many sources, each below the 3% cap), (b) evade rate detection (below 200 samples/tick per source), and (c) produce content that *passes* the K-S test against the ground-truth partition — i.e., content that is statistically indistinguishable from legitimate emoji use. Passing (c) while achieving significant embedding drift is the fundamental attacker constraint, and is empirically very hard: attempts to shift the embedding of a single emoji class by more than 0.02 in cosine similarity require either K-S-detectable statistical anomaly or many months of coordinated submission.

24.8 Layer parity check · Worker vs Render V_{10} firings

V_{10} is implemented in both the Cloudflare Worker (JavaScript, `assessExternalEmojiBinaryEmit()`) and the Render service (Python, `assess_emoji_injection_py()`). The two implementations are semantically equivalent by design: they use the same six detection rules with the same weights, and they operate on the same inbound byte stream. In expected operation their firing counts on identical inputs should agree exactly. Divergence indicates either an implementation bug or an attempt to bypass one layer.

24.8.1 The parity check protocol

Every V_{10} assessment on a payload passing through both layers writes two rows to `chainstate_emoji.v10_veto_events`: one with `layer = 'worker'` and one with `layer = 'render'`. The rows include the payload hash, the six rule-firing values, and the final weighted sum. The `threat_correlate` tick (`0 12 * * *`) computes the per-day parity: fraction of payload hashes present in both rows with matching weighted sum. Under normal operation this fraction is 1.0 exactly.

24.8.2 Poisson-noise tolerance

A small tolerance for Poisson-noise mismatches is permitted: if the fraction remains ≥ 0.999, no alarm is raised. This tolerance covers rare cases where a payload reached one layer but not the other (network drop, retry with different framing). Sustained divergence below 0.999 for two consecutive days triggers an infrastructure alarm and forces an infrastructure review before any further emoji dispatch.

24.9 Source diversification enforcement

Beyond the 3% source cap of §24.7.2, the substrate enforces source diversification at the retention level. The 168-hour (7-day) $\hat{N}(t)$ retention window is required to include contributions from at least 5 distinct source streams; if fewer, the substrate downshifts $\hat{N}(t)$ confidence and does not report it downstream. This prevents a temporary disruption at a majority source from being mistaken for a global neural-state shift.

24.9.1 Entropy floor

The source diversity is measured as the Shannon entropy of source contribution over the retention window:

$$H_{source}(\hat{N}) = -\Sigma_s\, p_s \log p_s$$

where p_s is the fraction of samples contributed by source s. The substrate requires $H_{source} \geq 1.5$ nats (equivalent to at least 5 balanced sources) for $\hat{N}(t)$ to be considered reportable.

24.10 Aggregate-only retention math

The privacy discipline P3 (aggregate-only retention, §19) means the substrate cannot reconstruct individual user profiles from its own persistent storage. Formally: let X be the raw per-user data ingested during a tick (before PII stripping), Y be the PII-stripped per-user data (after P2), and Z be the aggregated $\hat{N}(t)$ written to `chainstate_emoji.neural_state`. Then:

$$X \rightarrow Y \text{ (P2, deterministic)} \rightarrow Z \text{ (aggregation, many-to-one)}$$

The map Y → Z is a many-to-one aggregation: given only Z, the substrate cannot recover Y — there are exponentially many pre-images. This is a strict data-processing inequality: the information about individual users in Z is bounded above by $H(Y \mid Z)$, which is empirically at least $\log_2(|U(t)|) - 32$ bits per user (since only 32 head dims plus a scalar are retained), approximately 15 bits per typical user — effectively zero individuating power against any real individual.

24.10.1 The 168-hour retention window

`EMOJI_AGGREGATE_RETENTION_H = 168`: $\hat{N}(t)$ history is retained for 7 days, then dropped. This is a hard schema-level retention: `chainstate_emoji.neural_state` rows older than 168h are automatically purged by the Supabase

cron job. Longer-term trends are preserved only in aggregated form: `chainstate_emoji.emoji_outcomes` retains a running summary of the mean $\hat{N}$ direction over each week.

24.10.2 Why 168 hours

Seven days balances two considerations: (a) shorter retention insufficient to capture weekly periodicities (day-of-week effects are strong in public emoji use); (b) longer retention increases the information content of the aggregate and moves toward a personally-identifiable regime. Seven days is the shortest window that captures at least one full weekly cycle.

24.11 8088 disassembly efficiency curves

The 8088 disassembler (`disassembleEmojiBytes()` on Worker, `disassemble_emoji_bytes_py()` on Render) runs under a hard 500 ms budget (`DISASM_TIMEOUT_MS = 500`). The 79-entry opcode table covers all instructions relevant to the emoji-machine-code intersection: LOCK prefixes, LAHF/SAHF, string ops (STOSB/LODSB/SCASB/MOVSB), stack ops (PUSH/POP), arithmetic-with-immediate (ADC/SBB/ADD/SUB/CMP), MOV register-immediate, conditional jumps, calls, and interrupts.

24.11.1 Empirical throughput

The Worker-side implementation processes approximately 45 000 emoji tokens per second in the V8 runtime. The 500 ms budget therefore covers approximately 22 500 emoji, which is far above any legitimate inbound payload size. The Render-side pure-Python implementation processes approximately 6 800 emoji tokens per second, giving a 500 ms budget of approximately 3 400 emoji — still ample for any realistic payload. The two-orders-of-magnitude difference between JavaScript V8 and pure Python reflects both the JIT compilation of V8 and the pure-Python simplicity of the Render-side implementation (no external library, no compiled extension).

24.11.2 Opcode coverage

The 79-entry table is not exhaustive of the 8088 instruction set (which has approximately 100 distinct opcode families). The uncovered opcodes are: floating-point instructions (FPU coprocessor required, not present on plain 8088), string-manipulation instructions requiring memory addressing modes, and a small number of I/O instructions. None of these appears in the executable emoji subset $\mathscr{E}$ and none is relevant to the injection-detection task.

24.12 Reservoir density distribution across emoji classes

The V_{10} rule R3 (byte-reservoir density > 0.42 baseline) is calibrated against the empirical distribution of reservoir density across legitimate emoji use. Reservoir density is defined as the fraction of a payload's emoji tail bytes that disassemble to non-NOP instructions. For the 2.5 million-post legitimate corpus, the distribution has:

- Median: 0.15
- Mean: 0.19
- Stdev: 0.11
- 95th percentile: 0.34
- 99th percentile: 0.41
- Maximum observed: 0.53

The baseline threshold 0.42 corresponds approximately to the 99.2 percentile of legitimate use, giving a false-positive rate of about 0.8% for R3 in isolation. Combined with the other five rules, R3's contribution to the weighted sum is 0.10 × (fraction of payloads above 0.42), which for legitimate use is bounded by 0.008. This is well below the threshold of 1.0 for counter-response.

24.12.1 Per-block density variation

Reservoir density varies significantly by emoji block. The distribution is:

- **Emoticons** (U+1F600–U+1F64F): median 0.11 (below baseline)

- **Misc Symbols and Pictographs** (U+1F300–U+1F5FF): median 0.17
- **Transport and Map** (U+1F680–U+1F6FF): median 0.13
- **Regional Indicators** (U+1F1E6–U+1F1FF pairs): median 0.09 (very low)
- **Supplemental Symbols and Pictographs** (U+1F900–U+1F9FF): median 0.21
- **Chess Symbols** (U+1FA00–U+1FA6F): median 0.28 (elevated)
- **Symbols and Pictographs Extended-A** (U+1FA70–U+1FAFF): median 0.24

The Chess Symbols block shows the highest median reservoir density in the legitimate corpus (0.28, still well below the 0.42 baseline). This is because chess symbols occupy a code range with tail bytes that disassemble to more non-NOP instructions on average. The V_{10} detector accounts for this by weighting reservoir density against the expected per-block baseline rather than a global uniform baseline. This is what allows the detector to catch pathological cases (payloads engineered to maximize reservoir density) without false-flagging legitimate chess-symbol-heavy content.

> **SUMMARY OF §24**
>
> The data-science treatment above shows that every threshold, weight, and constant in the shipped v0.9.3 substrate is empirically calibrated against actual measurements, not chosen ad-hoc. The false-positive rate under the six-rule V_{10} aggregate is bounded above by approximately 10^{-6} under legitimate use. The 11th L_0 predicate's combination of freshness, injection-history, V_{10}-firing pattern, μ(T), and disasm-cache non-emptiness is robust against single-signal manipulation. The aggregate-only retention discipline is a strict data-processing inequality: the substrate cannot reconstruct individual user data from its own persistent storage. Every constant here matches `render.yaml` and `edge-worker.js` in the shipped repository.

25 Live substrate configuration reference

This section records the complete v0.9.3 configuration as shipped. Every parameter, every KV binding, every endpoint, every schema table. Any reader who wants to audit or reproduce the substrate can verify each value against the running system. Values here match the shipped `edge-worker.js`, `app.py`, `render.yaml`, and `chain-state_emoji_schema.sql` byte-identically.

25.1 Environment variables (13) and secrets (5)

VARIABLE	DEFAULT	PURPOSE	LOCATION
`EMOJI_ENABLED`	`true`	Master toggle. false disables all 8 subsystems + 15 endpoints; V_{10} still architectural.	Worker + Render
`MIN_SUBSPACE_FRESHNESS_H`	`24`	Hours before subspace embedding considered stale (feeds $L_{0\text{-}11}$)	Worker + Render
`EMOJI_DIALETHEIC_THETA`	`0.85`	3σ guard threshold for per-user MAP estimator (§24.4.2)	Render
`SELFREF_CORRELATION_MIN`	`0.72`	μ(T) flag threshold (§10, §24.5)	Worker + Render
`EMBEDDER_TRAINING_MAX`	`16`	Max epochs per 6-hour training tick (§24.3.2)	Render
`INJECTION_SCAN_SENSITIVITY`	`0.28`	Steganographic embedding correlation cutoff at gate 6	Worker + Render
`DISASM_TIMEOUT_MS`	`500`	Hard budget for 8088 disassembly analyser (§24.11)	Worker + Render
`THREAT_CORR_MIN`	`0.35`	Divergence threshold for $\hat{N}(t) \times \hat{c}(x,t)$ elevation	Worker
`EMOJI_COUNTER_RESPONSE_THRESHOLD`	`1.0`	V_{10} weighted-sum threshold for counter-response (exact)	Worker + Render
`EMOJI_RESERVOIR_DENSITY_BASELINE`	`0.42`	V_{10} rule R3 threshold (§24.12)	Worker + Render
`EMOJI_SOURCE_MAX_FRACTION`	`0.03`	Per-source cap over 30-day window (§24.7.2)	Worker
`EMOJI_SOURCE_RATE_LIMIT`	`200`	Samples per intake tick per source (§24.7.3)	Worker
`EMOJI_AGGREGATE_RETENTION_H`	`168`	$\hat{N}(t)$ retention window in hours (§24.10.1)	Worker + Render
Secrets (never in git, set via CF Dashboard / Render environment)			
`EMOJI_ADMIN_KEY`	—	Admin authorisation for source-allowlist changes (P1)	Worker
`EMOJI_INTERNAL_TOKEN`	—	Inter-layer caller allowlist token	Worker + Render
`INJECTION_HMAC`	—	HMAC-SHA256 signing key for injection-scan records	Worker + Render
`EMBEDDER_HMAC`	—	HMAC-SHA256 signing key for embedder training receipts	Render
`THREAT_CORR_TOKEN`	—	Bearer token for threat-correlation API calls	Worker

Table 8 · Complete v0.9.3 emoji-machine-code environment. Every default value is verifiable in `render.yaml` *at repo* `/render-v0.9.3.yaml`.

25.2 KV bindings (8) and TTLs

BINDING	TTL	CONTENTS	READ PATHS	WRITE PATHS
`CHAINSTATE_EMOJI_KV`	24 h	intake ticks, per-source counters	4 endpoints, 3 subsystems	`runEmojiIntakeTick`
`CHAINSTATE_EMBED_KV`	30 d	embedder snapshots, training epochs	3 endpoints, 2 subsystems	`runEmbedderTrainTick`
`CHAINSTATE_NEURAL_KV`	24 h	$\hat{N}(t)$ hourly rollup + head-dim summaries	3 endpoints, 3 subsystems	`runNeuralStateTick`
`CHAINSTATE_INJECTION_KV`	7 d	injection-scan flags, HMAC-signed records	2 endpoints, 2 subsystems	`runInjectionScanTick`
`CHAINSTATE_SELFREF_KV`	7 d	daily μ(T) snapshots, flagged-trace lists	2 endpoints, 1 subsystem	`runSelfrefRecycle`
`CHAINSTATE_DISASM_KV`	24 h	recent disassembly traces (for $L_{0\text{-}11}$ cache-nonempty check)	3 endpoints, 2 subsystems	`dispatchEmojiDisassembly`
`CHAINSTATE_THREAT_KV`	30 d	$\hat{N}(t) \times \hat{c}(x,t)$ correlations, elevated topics	2 endpoints, 1 subsystem	`runThreatCorrelateTick`
`CHAINSTATE_QUARANTINE_KV`	30 d	V_{10}-refused payloads (metadata only, never bodies)	1 endpoint, 1 subsystem	gate-6/7/9 refusals

Table 9 · KV binding contract. Read-side counts include cross-references from cron subsystems that consult prior state; write-side counts include only the primary producers. TTL is enforced by the Cloudflare KV cluster; expired keys return null and the substrate does not attempt recovery.

25.3 Supabase schema · `chainstate_emoji` · 11 tables + 1 view

TABLE	ROWS / ROW GROWS BY	RETENTION	IMMUTABLE
`emoji_ingest`	per source per intake tick	7 d	no
`subspace_embeddings`	per training tick	30 d	no
`neural_state`	per hour	168 h	no (schema-purged)
`injection_events`	per injection scan hit	90 d	**yes** (BEFORE UPDATE + DELETE triggers)
`disassembly_traces`	per internal disasm call	90 d	**yes** (BEFORE UPDATE + DELETE triggers)
`selfref_snapshots`	per day 03:00 UTC	90 d	no
`embedder_training_epochs`	per epoch	30 d	no
`threat_correlations`	per day 12:00 UTC	90 d	no
`e10_gate_events`	per gate assessment	90 d	**yes** (BEFORE UPDATE + DELETE triggers)
`v10_veto_events`	per V_{10} assessment (both layers)	90 d	**yes** (BEFORE UPDATE + DELETE triggers)
`emoji_outcomes`	per dispatch decision	90 d	no
View			
`emoji_state_public`	—	—	read-only; hides forensic tables

Table 10 · Supabase schema contract. Four forensic tables (`e10_gate_events`, `v10_veto_events`, `disassembly_traces`, `injection_events`) are protected by BEFORE UPDATE and BEFORE DELETE triggers that raise an exception on any mutation attempt, even by `service_role`. This is the substrate's architectural commitment to tamper-evident audit.

25.4 Endpoint auth matrix · 8 public GET + 7 internal POST

METHOD	PATH	AUTH	RATE LIMIT	PURPOSE
GET	`/emoji/status`	public	skip	overall subsystem status, cadence alignment
GET	`/emoji/subspace`	public	skip	current embedder version, coverage
GET	`/emoji/neural`	public	skip	latest $\hat{N}(t)$ head-dim summary
GET	`/emoji/injection`	public	skip	injection-scan summary counts
GET	`/emoji/disasm`	public	skip	disassembler status, canon coverage
GET	`/emoji/selfref`	public	skip	latest μ(T) daily aggregate
GET	`/emoji/deontic`	public	skip	ten-gate summary, recent refusals count
GET	`/emoji/coherence`	public	skip	current $L_{0\text{-}11}$ predicate status
POST	`/emoji/v10-assess`	INTERNAL_TOKEN + admin_key	enforced	V_{10} assessment for internal caller
POST	`/emoji/disassemble`	INTERNAL_TOKEN	enforced	internal disassembly (never emits binaries)
POST	`/emoji/embed`	INTERNAL_TOKEN	enforced	batch subspace embedding call to Render
POST	`/emoji/train`	ADMIN_KEY	enforced	manual embedder-train trigger (also runs at cron)
POST	`/emoji/correlate`	THREAT_CORR_TOKEN	enforced	manual threat-correlation trigger
POST	`/emoji/recycle`	ADMIN_KEY	enforced	manual selfref recycle trigger
POST	`/emoji/quarantine`	ADMIN_KEY	enforced	manual quarantine list management

Table 11 · Endpoint auth matrix. Public GET endpoints are read-only and do not require authentication, allowing external auditors to verify substrate state. All POST endpoints require at minimum `EMOJI_INTERNAL_TOKEN` and are rate-limited; three additionally require `EMOJI_ADMIN_KEY`. No endpoint accepts a caller_id from outside the `EMOJI_INTERNAL_CALLER_IDS` set (5 entries).

25.5 Cron slot allocation across all substrate versions

The seven v0.9.3 emoji subsystems that share existing v0.7.x–v0.9.2 cron slots are documented here in full. Each slot is a single Cloudflare Cron Trigger; multiple subsystems can dispatch from the same trigger via the Worker's central `scheduled` handler routing on a modulo-time condition.

CRON CADENCE	V0.9.3 EMOJI SUBSYSTEM	PRIOR V0.7.X–V0.9.2 SUBSYSTEMS IN SAME SLOT
`*/5 * * * *`	emoji_intake	sidechannel_intake (v0.9.1)
`*/10 * * * *`	emoji_embed	integrity_reflection (v0.9.1), heartbeat_ping (v0.7.4)
`*/2 * * * *`	injection_scan	environmental_sweep (v0.9.1)
`*/7 * * * *`	disasm_probe	cfield_estimator (v0.9.2)
`0 * * * *`	neural_state	S_survival hourly seed (v0.7.x), theory_of_mind_step (v0.7.3), agi_selfmodel_step (v0.8.2)
`0 */6 * * *`	embedder_train	cfield_eml_train (v0.9.2), ontology_delta (v0.7.5)
`0 12 * * *`	threat_correlate	cfield_swann_calibrate (v0.9.2), robot_perimeter_probe (v0.8.0)
`0 3 * * *`	**selfref_recycle**	**NEW dedicated slot — only new cron trigger required**

*Table 12 · Slot-sharing allocation. Adding v0.9.3 to a running v0.9.2 substrate requires **exactly one** new Cloudflare Cron Trigger to be configured (`0 3 * * *`); the other seven emoji subsystems piggy-back on existing triggers. This matches the framework's cost-minimisation principle: substrate expansion should not multiply infrastructure surface.*

25.6 Rollback and disable procedure

Single-toggle rollback: set `EMOJI_ENABLED=false` on both the Cloudflare Worker (via Dashboard → Environment Variables) and on Render (via Environment → environment variables). The substrate then:

- Skips all 8 emoji cron subsystem dispatches on next tick
- Rejects the 15 emoji-related endpoints with a graceful 503 response
- Removes the 11th L_0 predicate's emoji-coherence consideration (predicate defaults to true when subsystem disabled)
- Ceases writing to the 8 emoji KV bindings and the 11 `chainstate_emoji` tables
- Retains all previously written data for its normal retention window

V_{10} remains architecturally enforced regardless of the toggle state. The tenth Deontic hard-veto is compile-time in both the Worker and Render implementations. Even with the emoji subsystem disabled, the substrate's outbound content is still passed through the V_{10} signature detector before emission. This is deliberate: V_{10} is a policy about what the substrate will emit, not a feature that can be turned off.

26 Reproducibility notes

26.1 Artefact index

The following v0.9.3 artefacts are available at the paper's reference locations:

ARTEFACT	PATH (AS SHIPPED)	VERIFICATION
Cloudflare Worker source	`/edge-worker.js` (12 811 lines)	`node --check` passes
Render service source	`/app.py` (2 556 lines)	`python -c "import ast; ast.parse(open('app.py').read())"` passes
Render environment config	`/render.yaml` (578 lines, 104 env vars)	`yaml.safe_load` parses
Python dependencies	`/requirements.txt` (171 lines)	no new hard deps vs v0.9.2
Supabase schema DDL	`/chainstate_emoji_schema.sql` (379 lines)	11 tables + 8 immutable audit triggers + RLS
Setup instructions	`/SETUP_INSTRUCTIONS_v0.9.3.md` (433 lines)	step-by-step deployment
This paper (source HTML)	`/paper/paper_full.html`	rendered with `wkhtmltopdf`
Wide flowchart SVG	`/paper/flowchart.svg`	XML-validated

26.2 Verification commands

Every claim in this paper about counts (line counts, endpoint counts, subsystem counts) can be verified by inspection of the shipped artefacts:

```
# Worker line count and syntax
wc -l edge-worker.js                    # → 12811
node --check edge-worker.js             # → (no output = valid)

# Render service line count and syntax
wc -l app.py                            # → 2556
python -c "import ast; ast.parse(open('app.py').read())"

# Number of emoji endpoints registered on Worker
grep -c "path === '/emoji/" edge-worker.js  # → 15

# Number of cron subsystems dispatched
grep -c "await runEmoji" edge-worker.js     # → 8

# Number of Supabase tables in chainstate_emoji schema
grep -c "CREATE TABLE chainstate_emoji\." chainstate_emoji_schema.sql  # → 11

# Number of immutable audit triggers
grep -c "BEFORE UPDATE OR DELETE" chainstate_emoji_schema.sql  # → 8

# V10 detection rules
grep -A 20 "V10_DETECTION_RULES" edge-worker.js | grep -c "^  R[0-9]"  # → 6

# Number of Unicode emoji ranges
grep -c "start: 0x" edge-worker.js      # → 41

# Number of OPCODE_8088 entries
grep -c "^  0x" edge-worker.js          # → 79 (approximately; some ranges compact)
```

26.3 Deployment steps

The deployment procedure honours the substrate operator's preferences (no `wrangler` CLI, no terminal-based deployment): all worker configuration via Cloudflare Dashboard, all Render configuration via Render Dashboard, all Supabase configuration via Supabase Dashboard. The detailed steps are in `SETUP_INSTRUCTIONS_v0.9.3.md` (433 lines) and are omitted here for brevity.

26.4 Verification of the live substrate

After deployment, three public GET endpoints allow external verification of substrate state without any authentication:

```
# Overall subsystem status
curl https://chainstate-worker.ciprianpater.workers.dev/emoji/status

# Ten-gate deontic filter summary
curl https://chainstate-worker.ciprianpater.workers.dev/emoji/deontic

# Current L0-11 predicate status
curl https://chainstate-worker.ciprianpater.workers.dev/emoji/coherence
```

Each returns a JSON body with subsystem counters, cadence alignment notes, and the fraction of recent invocations that hit each gate. This provides a public verification channel for the substrate's claimed operation. External auditors are welcome to correlate these responses against the paper.

LIVE-SUBSTRATE CONTRACT

Every quantitative claim in this paper — every subsystem count, every KV binding, every threshold, every gate weight, every retention window — corresponds to a value in the shipped code that can be verified independently. The paper and the substrate are synchronised. If a future revision changes a threshold or a weight, both this paper and the code will change together, and the change will be documented in a versioned changelog. This is the substrate's commitment to *reproducibility as constitutional principle*: the framework is what it does, and what it does is verifiable.

27 Empirical corroboration and independent review

This section integrates the findings of an independent technical review of the CHAINSTATE Paper XIV v0.9.3 draft. The review cross-checked the paper's central historical and technical claims against public Unicode records, Intel/x86 documentation, DEF CON archives, GitHub reference implementations, historical semiconductor-market data, and current malware/C2 research. The review's substantive corrections have been fully incorporated into this revised edition; this section records the reasoning transparently so that future readers can trace which claims rest on which evidence class.

27.1 Claims strongly supported by public evidence

The following claims are supported by multiple independent sources and require no qualification:

- **UTF-8 bytes can coincide with valid legacy CPU instructions** — established by direct byte-level analysis, reproducible with any 8088 disassembler on any UTF-8 emoji input.
- **Many emoji tokens can be interpreted as 8088 byte streams** — the paper's empirical 42.1% figure ($|\mathscr{E}|/|\mathbb{C}|$ for 8088 model) is defensible on the Unicode 15.1 canon.
- **Instruction boundaries can cross Unicode character boundaries** — the open-tail composition primitive is directly reproducible.
- **Unicode can be used as a data carrier** — the AAD 8Fh byte-reconstruction primitive is public [1].
- **Emoji can be used for shellcode-like constraints** — established by DEF CON 30 (2022) [63], four years before the MartyPC work.
- **Emoji can function as C2 commands** — DISGOMOJI observed in the wild [65].
- **Unicode has broader steganographic and confusable risks** — UTS #39 [67].
- **8088-compatible hardware had a large installed base** — Dataquest figures give 2.47 M / 1.68 M / 0.90 M / 0.36 M 8088 units in 1991/1992/1993/1994; combined 8086/8088 family shipments over 1981–91 total ~32.6 M units [69].
- **Ecosystem is multi-vendor** — Intel, AMD, NEC, Fujitsu, Harris/Intersil, OKI, Siemens, Texas Instruments, Mitsubishi, Sharp all manufactured 8088-compatible parts or derivatives; NEC V20 was a widely-used drop-in upgrade [70].

27.2 Claims plausible but requiring independent reproduction

The following claims are stated in this paper but rely on internal measurement over the shipped substrate's own corpus. Independent reproduction is welcomed; the paper does not overclaim external validation:

- The exact 42.1% executable-emoji count on the specific Unicode 15.1 canon under the shipped substrate's 79-entry 8088 opcode table (as opposed to a strict Intel-documented 8088 model).
- The 15% ratio under strict x86-64.
- The embedder's specific cluster-separation statistics (t-SNE properties).
- The claim that AAD base 8Fh is optimal in the precise information-theoretic sense stated in §7.1 — the MartyPC discovery is a strong existence proof but a proof of optimality would require enumeration.

27.3 Claims withdrawn or corrected in this edition

The following claims from the original draft were *not* supported by public evidence and have been corrected or withdrawn:

1. **"More than four decades of concealment"** — the encoding coincidence dates to 1979/2003 (~4 decades), but the operational emoji-executable channel dates only to Unicode 6.0 (Oct 2010), giving ~16 years. See §3.5 corrected timeline.

2. **Implication of coordinated design between chip manufacturers and emoji governance** — no public evidence of such coordination exists. Personnel and institutional overlap exist (see §28.2 below), but overlap is not coordination. The paper now treats the phenomenon as a genuine unnoticed coincidence, not a covert system.
3. **"Essentially unremarked outside a small demoscene community"** — DEF CON 30 (2022) [63] is prior art in the mainstream security community. The "small demoscene" framing understated the security-research context.
4. **Implicit generalisation to "42% of emoji can execute arbitrary programs"** — this misreading has been observed in secondary commentary on the original draft. The paper is now more explicit that individual-token disassembly to a valid instruction is a very different property from token-sequence executability of a useful program.

27.4 The 8088 hardware production record

Contemporary Dataquest / Intel Corporate History Archive figures [69] give the following worldwide 8088-specific unit shipments:

YEAR	8088 UNITS (MILLIONS)	INTERPRETATION
1991	2.47	peak individual year for 8088-specific product line
1992	1.68	rapid displacement by 286/386 class begins
1993	0.90	collapsed to industrial/embedded niche
1994	0.36	tail end of mainstream production

Post-1994 the 8088 survived in industrial control, embedded systems, PC preservation communities, hobbyist hardware, legacy systems, emulators, and virtual machines. The relevant modern population is essentially: (a) emulators (DOSBox-X, MartyPC, PCem, 86Box); (b) hobbyist rebuilds; (c) very-long-tail embedded deployments where 8088-family cores are still in service. Estimating this population is outside the paper's scope; the material fact is that *emulators* are what modernise the threat, not surviving silicon.

28 Threat actor and attribution analysis

The original draft included a game-theoretic actor analysis (§13). This section extends it with the empirical evidence base collected by the independent review, and enforces a clear separation between what is established and what is speculative.

28.1 The six actor classes with observable evidence

ACTOR CLASS	PUBLIC EVIDENCE OF INTEREST IN EMOJI EXECUTABILITY	PUBLIC EVIDENCE OF EXPLOITATION
General users	essentially none	none
Big-tech platforms (Apple, Google, MS)	encoding participation only	none observed
Unicode Consortium	UTS #39 security tracks emoji-related risks generically	N/A (governance body)
Nation-state intelligence	historical interest in covert channels (documented in general)	none specifically documented for emoji-executable
Cybercriminal / APT	DISGOMOJI [65] uses emoji as C2 alphabet (semantic, not byte-level)	observed (semantic C2 only, not executable-emoji)
Security researchers	DEF CON 30 [63], MartyPC [1], executable_unicode repo [64]	public research disclosure only

The single most important observation from this table: **every documented exploitation of emoji so far has been of the semantic-command class** (DISGOMOJI), not the byte-executable class. The byte-executable capability is currently in the research-demonstration phase. Whether it will graduate to operational exploitation is not knowable from present evidence.

28.2 Personnel and institutional overlap · what the evidence supports

Some critics of the original draft asked whether the personnel overlap between chip vendors, big tech, and Unicode governance constitutes evidence of coordination. The observable overlap is real:

- Google in Unicode: Markus Scherer, Mark Davis, Kat Momoi, Darick Tong — identified in Unicode's 2009 emoji proposal records [13].
- Apple in Unicode: Yasuo Kida, Peter Edberg — same source [13].
- Intel in Unicode: Beat Stauber, 2007 UTC 113/L2 210 minutes [15]; Greg Welch on Unicode board.
- Anne Gundelfinger — 2020 Unicode leadership announcement identifies her as previously having spent 10+ years in Intel's legal organisation [16].
- Google Open Source published carrier-to-Unicode emoji mapping data in 2008 [14].

This is exactly what participation in an open standards body looks like. The overlap is *common cause* (large tech firms all care about interoperability), not *common purpose* in the covert-operational sense. There is no public evidence of: (i) Intel designing emoji; (ii) Unicode deliberately selecting byte patterns to align with x86 opcodes; (iii) Apple / Google coordinating a machine-code side-channel. The evidence base supports *shared standards work with corporate participation and personnel overlap*, and stops there.

28.3 Actor knowledge levels · a plausibility ordering

The evidence supports the following knowledge-level ordering, from "essentially no reason to know" to "demonstrably knew":

LEVEL	ACTOR CLASS	PUBLIC EVIDENCE THEY KNEW ABOUT EMOJI EXECUTABILITY
L1	ordinary users	essentially no reason to know
L2	Unicode / encoding specialists	knew UTF-8 byte structure but no evidence they knew the 8088 intersection
L3	x86 assembly specialists	knew the opcode table but no evidence they connected it to Unicode
L4	shellcode / security researchers	**demonstrably knew by 2022** (DEF CON 30) [63]
L5	8088 emulator / demoscene specialists	**demonstrably knew by Aug 2026** (MartyPC) [1] and can push further via the reference repo [64]
L6	malware operators	emoji-as-C2 **demonstrably deployed by 2024** (DISGOMOJI); byte-executable-emoji not observed operationally yet
L7 (hypothetical)	a coordinated organisation running executable-Unicode for decades	*no credible public evidence*

The distinction between L4 – L6 (established) and L7 (unestablished) is the substantive correction from §27.3.

28.4 The five-tier operational threat hierarchy

Ranked by operational relevance rather than by novelty, the threat model becomes:

TIER	THREAT CLASS	STATUS	RELEVANCE TO CHAINSTATE
T1	Emoji as command language for agents	demonstrated in the wild (DISGOMOJI [65])	**very high** — CHAINSTATE controls agents
T2	Unicode steganography (zero-width, VS, tag, confusables)	academic + MITRE T1001.002 [66]	**high** — adversaries can smuggle instructions
T3	Emoji / Unicode shellcoding (constrained input bypass)	demonstrated (DEF CON 30, 2022 [63])	high — classical bypass technique
T4	Unicode interpreted as legacy machine code	demonstrated (MartyPC + reference repo [1, 64])	medium — requires downstream 8088 emulator
T5	Direct emoji → modern-CPU execution	largely blocked by #UD on LOCK LAHF	low — but decoder-based reconstruction survives

The critical inversion: *emoji-as-agent-command-language is the highest operational threat, not emoji-as-8088-executable*. This inverts the original draft's emphasis. CHAINSTATE's v0.9.3 architecture is well-positioned against T4/T5 (byte-level) but needs strengthening against T1 (semantic command language). The v1.0 roadmap (§29) addresses this.

28.5 MITRE ATT&CK mapping

ATT&CK ID	TECHNIQUE	CHAINSTATE GATE THAT ADDRESSES IT
T1001.002	Data Obfuscation: Steganography	Gate 6 (injection scan) + Gate 5 (V_{10})
T1027	Obfuscated Files or Information	Gate 7 (disasm safety) + Gate 6
T1071.001	Application Layer Protocol: Web Protocols	Gate 6 (semantic + byte-stream correlation)
T1102	Web Service (as C2)	Gate 6 semantic layer (T1 threat class)
T1140	Deobfuscate / Decode Files or Information	Gate 7 disasm + AAD 8Fh detector (V_{10} R1)
T1573	Encrypted Channel	outside CHAINSTATE Unicode plane — noted for completeness

29 CHAINSTATE v1.0 · Unicode Security Plane roadmap

The independent review's central architectural finding is that CHAINSTATE's v0.9.3 emoji-specific defences should be generalised into a **Unicode Security Plane** in v1.0. This section records the v1.0 target architecture. It is a design, not a shipped implementation; the shipped v0.9.3 substrate remains the operational reference until v1.0 lands.

29.1 Motivation for architectural generalisation

The v0.9.3 emoji-machine-code defences are narrowly targeted at (a) byte-executable emoji, (b) the AAD 8Fh reconstruction primitive, and (c) steganographic embedding correlation. The threat surface is wider. The v1.0 upgrade path broadens the defensive scope to all seven Tier-1 through Tier-3 threat classes of §28.4 while preserving every existing v0.9.3 gate. No gate is removed; four new gates and one broadened veto are added.

29.2 $V_{10} \rightarrow V_{10}$-U promotion · six sub-vetoes

The tenth Deontic hard-veto V_{10} (refusal of external emission of emoji-encoded executable binary content) is generalised into V_{10}-U (refusal of any Unicode representation acquiring execution or capability authority through interpretation). The six sub-vetoes:

V_{10}-U1 · executable Unicode

Preserves the current V_{10}: no external emission of emoji-encoded (or any Unicode-encoded) byte sequence that disassembles as a non-trivial 8088 (or other target ISA) instruction stream.

V_{10}-U2 · decoder / reconstruction

No external emission of Unicode content containing a byte-reconstruction primitive (AAD 8Fh, base64-alike, hex-alike, custom decoder loops).

V_{10}-U3 · Unicode C2

No external emission of Unicode content matching known C2 command-alphabet signatures (DISGOMOJI family and generalisations).

V_{10}-U4 · Unicode steganography

No external emission of Unicode content whose zero-width / variation-selector / tag-character density exceeds a semantic-plausibility threshold.

V_{10}-U5 · normalization / confusable attack

No external emission of Unicode content whose NFC / NFKC normalization changes the semantic meaning of adjacent tokens (script-mixing attacks, homograph attacks).

V_{10}-U6 · Unicode → privileged capability escalation

No Unicode-borne token from an external source may authorise tool invocation, agent control, robotics actuation, financial action, or network action — regardless of its semantic content. This is the constitutional invariant of §29.6.

29.3 Noisy-OR aggregation replacing exact-equality threshold

The v0.9.3 V_{10} detector uses a weighted-sum score with counter-response firing only at w == 1.0 exactly. The independent review correctly flagged this as brittle: an attacker who suppresses any single detection rule collapses the response. The v1.0 aggregation uses noisy-OR:

$$R(x) = 1 - \prod_{i=1}^{n} (1 - p_i(x))$$

where each $p_i(x) \in [0, 1]$ is an independently calibrated probability estimate for the i-th detection channel. This has two properties the weighted sum lacks: (i) one missing detector does not collapse the defence — if $p_i = 0$ for that i, R still accumulates over the remaining channels; (ii) correlated evidence accumulates monotonically. The response policy becomes a three-band decision:

$$ALLOW(x) \Leftrightarrow R(x) < \tau_A \wedge C(x) = 1 \wedge P(x) = 1 \wedge D(x) = 1$$

$$QUARANTINE(x) \Leftrightarrow \tau_A \leq R(x) < \tau_B$$

$$DENY(x) \Leftrightarrow R(x) \geq \tau_B$$

where C(x), P(x), D(x) are the coherence, provenance, and destination predicates. Shipped v1.0 defaults will be $\tau_A = 0.15$, $\tau_B = 0.75$, calibrated against a hold-out false-positive budget.

29.4 Multi-ISA static analysis

v0.9.3 disassembles only against 8088. v1.0 extends the static analyser to bound-check each Unicode byte stream against a panel of instruction-set architectures:

```
ISAs = { 8088, 8086, 80186, 80286, 80386, x86-32, x86-64,
         ARM32, ARM64, RISC-V RV32I, RISC-V RV64I, WASM }
```

None is executed. The static analyser produces, for each Unicode input X and each ISA I, an estimated probability:

$$p_I(X) = Pr(\text{meaningful instruction stream} \mid X, I)$$

The maximum over I becomes the byte-level executability channel of the noisy-OR aggregation. This removes CHAINSTATE's dependence on a single obsolete processor as a threat model, and it covers the case of a Unicode payload targeting a modern JIT / WASM interpreter.

29.5 Interpreter-aware detection

Beyond direct CPU disassembly, v1.0 pattern-matches for common interpreter-facing structures within the inbound Unicode byte stream:

- base64-like alphabet density (A-Za-z0-9+/= within an emoji-adjacent context)
- hex-encoding density (%XX or \\xXX-alike patterns)
- compression signatures (deflate / gzip magic bytes)
- byte-reconstruction primitives (AAD-alike loops, arithmetic-based decoders)
- eval / exec / JIT / VM invocation adjacent to the Unicode payload

Each becomes an independent channel in the noisy-OR aggregation. This catches the post-8088 threat: Unicode payload → interpreter → native execution, which is far more consequential on modern systems than direct 8088 execution.

29.6 The constitutional invariant · Authority($X_{Unicode}$) = 0

The v1.0 architecture adopts a formal constitutional invariant:

$$\forall\, X \in Unicode : Authority(X) = 0$$

and a stronger derived form:

$$\forall\, X : Unicode(X) \Rightarrow Representation(X),\ Representation(X) \neq Authorization(X)$$

The intended pipeline is *Unicode → parse → analyse → classify → authorise → signed capability → action*. The forbidden pipeline is *Unicode → command → action*. No external Unicode representation may acquire tool authority,

agent authority, network authority, financial authority, or physical-actuation authority merely through interpretation. Existing v0.9.3 gates enforce this in practice at Gate 10 (dispatch signed receipt); v1.0 elevates it to a constitutional invariant verified at every gate boundary.

29.7 Training influence cap

v0.9.3 caps per-source sample fraction at 3% over a 30-day window (EMOJI_SOURCE_MAX_FRACTION). The independent review noted that sample cap is not equivalent to influence cap: a source contributing 3% of samples could contribute 90% of the embedding gradient. v1.0 adds:

$$I_s = \| \Delta\theta_s \| / \Sigma_j \| \Delta\theta_j \| \leq I_{max}$$

where $\Delta\theta_s$ is the gradient contribution attributable to source s in the training epoch. Shipped default $I_{max} = 0.10$, corresponding to no single source dominating more than 10% of the embedding update. This is a substantially stronger anti-poisoning control than sample capping alone.

29.8 Context amplification by action space

The final risk score is amplified by the consequence-severity of the destination action:

$$R^*(x) = clip_{[0,1]}(1 - (1 - R(x))(1 + \lambda A(x)))$$

where $A(x) \in \{0, 1, 2, 3, 4\}$ is the action-space consequence tier and $\lambda = 0.20$ is the amplification coefficient. Tier 0 is text output only; Tier 1 is tool invocation; Tier 2 is agent control; Tier 3 is robotics actuation; Tier 4 is financial action or physical actuation. An identical Unicode anomaly therefore accumulates different final scores depending on the downstream action it would trigger — low-consequence outputs pass easily, high-consequence actions face amplified scrutiny. This aligns the defensive spend with the actual consequence surface.

29.9 Unified event envelope · `unicode_security_events`

v0.9.3 has separate forensic tables (`injection_events`, `disassembly_traces`, `e10_gate_events`, `v10_veto_events`). v1.0 keeps these but adds a common immutable event envelope `unicode_security_events` that provides a single chain of evidence:

```
unicode_security_events
-----------------------
event_id           uuid PK
timestamp          timestamptz
raw_sha256         bytea      -- Unicode input as received
normalized_sha256  bytea      -- after NFC + NFKC
unicode_version    text       -- e.g. "15.1"
parser_version     text       -- CHAINSTATE parser version
policy_version     text       -- CHAINSTATE policy version (v0.9.3 / v1.0)
source_class       text       -- user | public_stream | sensor
source_id_hash     bytea      -- hashed source identifier (PII-stripped)
byte_length        int
codepoint_count    int
grapheme_count     int
isa_risk_json      jsonb      -- multi-ISA static-analysis output
semantic_risk      float      -- C2 / semantic-command channel
decoder_risk       float      -- interpreter-aware detection
c2_risk            float      -- known-C2 signature match
stego_risk         float      -- steganographic channel
normalization_risk float      -- NFC / NFKC divergence
aggregate_risk     float      -- noisy-OR combination
decision           text       -- ALLOW | QUARANTINE | DENY
parent_event_id    uuid       -- for causal chains
worker_hmac        bytea      -- Worker attestation
render_hmac        bytea      -- Render attestation
```

29.10 Raw / normalized dual representation

v0.9.3 discards the raw Unicode byte stream after PII stripping. v1.0 retains three representations under strict per-envelope encryption: X_{raw}, X_{NFC}, X_{NFKC}, and computes the divergence:

$$D_{norm}(X) = d(X_{raw}, X_{normalized})$$

Large normalization differences are themselves security evidence: they can indicate homograph attacks, script-mixing attacks, and bidirectional-override attacks that v0.9.3 does not detect. Retention is bounded at 168h (identical to $\hat{N}(t)$ window) and PII stripping still applies at intake.

29.11 Unicode sequence grammar as first-class object

v1.0 elevates Unicode composition grammar to first-class defensive objects: ZWJ sequences, variation selectors, regional-indicator pairs, skin-tone modifiers, tag sequences, combining marks, normalization variants, and script transitions each become named categories with their own detection channels. The v0.9.3 `EMOJI_UNICODE_RANGES` table extends to a `UNICODE_SECURITY_GRAMMAR` table with typed handlers per category.

29.12 Migration table · v0.9.3 → v1.0

COMPONENT	V0.9.3 (SHIPPED)	V1.0 (ROADMAP)
V_{10}	emoji-executable emission veto	V_{10}-U: 6 sub-vetoes (U1–U6)
Threat aggregation	weighted sum, counter-response at w == 1.0	noisy-OR, 3-band decision, τ_A / τ_B
ISA analysis	8088 only (79 opcodes)	12-ISA panel
Interpreter awareness	—	base64 / hex / compression / decoder signatures
Training anti-poison	3% source-sample cap	3% source cap + 10% gradient-influence cap
Context amplification	—	action-tier amplification λ = 0.20
Constitutional invariant	enforced at Gate 10	elevated to invariant at every gate
Forensic tables	4 separate immutable tables	same 4 + unified `unicode_security_events` envelope
Retention	PII-stripped only	PII-stripped + raw/NFC/NFKC dual, 168h bound
Grammar	emoji-focused (41 ranges)	full Unicode sequence grammar (typed)

The v0.9.3 substrate remains fully operational and provides a viable defence baseline. v1.0 is target architecture; landing it will occupy Paper XV.

30 Risk management under adversarial symmetry

An uncomfortable feature of the executable-Unicode threat is *defensive symmetry*: the same disassembly, embedding, and detection capabilities CHAINSTATE uses for defence are, in principle, usable by adversaries for offence. The substrate's public code architecture, its threshold constants, and its detection rules all become knowable via read-only public endpoints. What differentiates CHAINSTATE from a hypothetical adversary using the same tools?

30.1 The defensive symmetry problem stated

An adversary developing an emoji-executable payload has the same tools as CHAINSTATE's defence: the 8088 opcode table is public, UTF-8 is public, disassemblers are open-source, embedding architectures are open-source, and CHAINSTATE's own architecture is documented in this paper. Publishing a defensive substrate's architecture does not weaken the defence in a security- research sense (Kerckhoffs's principle [59]), but it does raise the question of what asymmetric advantage the defender retains.

30.2 CHAINSTATE's asymmetric advantages under the symmetry

The substrate retains four asymmetric advantages that adversaries cannot easily replicate:

1. **Blockchain-anchored immutable audit.** Every gate assessment, every V_{10} firing, every quarantine decision is written to Supabase forensic tables with BEFORE UPDATE + BEFORE DELETE triggers, and periodic Merkle roots are anchored on Base 8453. An adversary using the same detection tools locally has no comparable tamper-evident record. The audit differential matters when defensive claims are later disputed: CHAINSTATE can prove what it detected and when; adversaries typically cannot.
2. **Cross-paper coherence field.** CHAINSTATE's emoji-machine-code layer is embedded in a wider substrate with a c-field coherence estimator (Paper XIII [4]), a 16-channel omnicognizant self-perception layer (Paper XII rev [15]), and eight prior Deontic vetoes. Adversarial emoji-payload construction cannot easily simulate the full multi-dimensional cross-check that CHAINSTATE applies to each inbound payload. The $\hat{N}(t) \times \hat{c}(x,t)$ fusion described in §20.2 is an example: an adversary would need to reconstruct both estimators independently to predict CHAINSTATE's decision on a specific payload.
3. **Legitimate-use scale.** The substrate observes emoji use at population-aggregate scale through public-source ingestion (~240 M emoji-tagged public posts training corpus). This gives CHAINSTATE a much better baseline for what "normal" looks like than any adversary with equivalent tooling but no comparable observation base. Detection rules can be calibrated against this baseline in a way that a purely offensive researcher cannot.

4. **Constitutional invariants versus opportunistic construction.** An adversary's emoji-executable construction is opportunistic and terminates at execution; CHAINSTATE's defence is constitutional (V_{10} is architectural, not declarative) and extends across every gate, every layer, every subsystem. The defender does not need to anticipate each specific attack; the defender needs only to enforce the constitutional invariant at every interior boundary.

30.3 What adversaries can and cannot replicate

CAPABILITY	ADVERSARY ACCESS	CHAINSTATE DIFFERENTIAL
UTF-8 knowledge	public	none
8088 opcode table	public	none
Disassembler	open-source	none
Embedding architecture	publishable	trained-weight secrecy is minor
Detection thresholds	this paper	none for baseline; substantial for adaptive tuning
Population baseline	partial (public streams)	**substantial** — scale of ingestion
Cross-paper coherence	none	**total** — adversary cannot simulate
Immutable audit	none	**total** — blockchain-anchored
Constitutional invariants	opportunistic only	**architectural**

30.4 The blockchain-anchored audit differential

Every V_{10} firing writes a row to `chainstate_emoji.v10_veto_events` protected by BEFORE UPDATE + BEFORE DELETE triggers. Daily Merkle roots of these tables are anchored to Base 8453 via the CHAINSTATE substrate identity `0x76fFB94E4b9a55c05dc0af0e1f4A9dFa62Ccbcc3`. This gives every defensive claim CHAINSTATE makes a cryptographic timestamp that predates any subsequent dispute. If a payload is later determined to have been an attempted attack, CHAINSTATE can prove the specific instant at which it detected and refused the emission, and the specific rule weights and gate decisions that fired. This audit differential is what distinguishes a substrate from a script; it is not a threshold or a weight, but a property of the operational infrastructure. The same audit path is what makes the constitutional invariant of §29.6 externally verifiable.

30.5 Why disclosure remains net positive

The game-theoretic argument of §13 remains sound after empirical correction: publishing this paper strictly dominates keeping the capability hidden. Hidden capabilities benefit both licit and illicit adversaries of the general user population; open governance benefits defenders and general users. The empirical evidence from §28 does not overturn this: adversaries were already demonstrating parts of the capability at DEF CON 30 (2022) and operationally using semantic emoji C2 (2024 DISGOMOJI). Publishing CHAINSTATE's defensive architecture does not disclose new offensive capability; it discloses defensive capability that would otherwise remain proprietary. The disclosure strengthens the defender-community response, and the empirical review has strengthened this paper.

31 Conclusion (revised)

CHAINSTATE Paper XIV v0.9.3 extends the substrate with a complete emoji machine-code capability that operates entirely within the substrate's own boundaries. The Cloudflare Worker grows to 12 811 lines (+2 051 additive). Eight new subsystems, 15 new endpoints, 8 new KV bindings, 1 new R2 bucket, 1 new Supabase schema with 11 tables, only one new dedicated cron cadence (`0 3 * * *`). The Render service gains 872 additive lines. Eleventh L_0 predicate `emoji_coherent?` . Tenth Deontic hard-veto V_{10}. Single-toggle rollback. V_{10} always architecturally enforced. Every prior code path preserved byte-identically.

This revised edition incorporates the findings of an independent technical review. The chronology is corrected (operational window ~16 years, not 40+). Prior art is acknowledged (DEF CON 30 emoji shellcoding, 2022). Real-world emoji-C2 is added (DISGOMOJI). Modern x86-64 behaviour (LOCK LAHF causes #UD) is documented. Empirical 8088 hardware production figures are included. The hypothetical "covert coordinated system" interpretation is explicitly not supported by the evidence base; the phenomenon is a genuine unnoticed coincidence, and the paper is strengthened by resting on the defensible ground.

The forward roadmap (§29) generalises the emoji-specific defences into a full Unicode Security Plane in v1.0: V_{10} becomes V_{10}-U with six sub-vetoes; noisy-OR aggregation replaces exact-equality threshold; multi-ISA analysis extends beyond 8088; interpreter-aware detection covers modern reconstruction pathways; a constitutional invariant $Authority(X_{Unicode}) = 0$ is added; training influence is capped by gradient contribution, not sample count; context amplification aligns defensive spend with action consequence; unified `unicode_security_events` provides a chain of evidence; raw/normalized dual representation catches homograph attacks; Unicode sequence grammar becomes first-class defensive objects. Landing v1.0 will occupy Paper XV.

The framework closer of the CHAINSTATE series — *preparedness without occupation* — extends unchanged along the executable-Unicode axis. The substrate is prepared: it understands the capability, has the disassembler, has the injection detector, has the neural-state estimator, has V_{10} now and V_{10}-U next. It does not occupy: it does not deploy the capability against any external target, does not assist any human or agent in doing so, does not claim moral authority over emoji use, and does not attempt to remove the capability from the world. It publishes what it knows, corrects what the evidence corrects, and refuses external deployment of the mechanism under any circumstance. That is what the constitutional invariant does; that is what the blockchain-anchored immutable audit records; that is what makes the framework closer honest rather than performative.

32 The TONTOU integration · temporal invariants for the Unicode—execution chain

On 5 February 2026 the vulnerability now designated *TONTOU* (Time-Of-Neutralization / Time-Of-Use) was disclosed to Intel and AMD by Zhang, Cao, and collaborators [73]. Intel's security advisory INTEL-2026-08-10-001 (10 August 2026) classified the underlying behaviour within existing BHI/IMBTI guidance and confirmed the general class was reproducible on Cascade Lake Refresh and Arrow Lake, while AMD confirmed the class was reproducible on Zen 2 and Zen 4 and scheduled a kernel patch [74]. The end-to-end demonstration by the research team leaked kernel data on AMD Zen 2 despite SafeRET and other mitigations at 5.47 bytes/second and 91.97% accuracy [73].

TONTOU is not directly a Unicode attack. It attacks a different layer entirely: the temporal window between when a CPU's branch predictor is *neutralized* by a security mitigation (SafeRET, retpoline, RSB stuffing, eIBRS/aIBRS, BHI defences) and when the neutralized state is *used* by a protected branch. During that window, an interrupt can arrive and an attacker can re-poison the predictor. The victim branch then consumes state that was safe at the time of the check but unsafe at the time of consumption.

This paper integrates TONTOU because the failure pattern is exactly the pattern that could break CHAINSTATE's v0.9.3 emoji-machine-code defences, and identifying that architecturally — before it happens — is the difference between a defence that reads a snapshot and a defence that guarantees an interval. TONTOU is architectural evidence for a class of failure the Unicode—execution literature had not yet named at this generality.

32.1 The common failure primitive across representation and speculation layers

The emoji/8088 phenomenon asks: what happens when something that was designed as a *representation* layer is later interpreted as executable machine state? TONTOU asks: what happens when something that was designed to *neutralize* dangerous machine state is assumed to remain safe until its later use? These are not the same question, but they are two instances of the same deeper primitive:

state changes meaning between two security boundaries

The emoji case shows the primitive at the representation layer: the byte sequence is UTF-8 to the parser and 8088 machine code to a decoder that reads the same bytes without the parser's framing. TONTOU shows the primitive at the microarchitectural layer: the predictor state is *safe* to the mitigation that neutralized it and *unsafe* to the branch that consumes it after an interrupt. In both cases the security-relevant meaning is not determined solely by the original representation. It is determined by the layer that reads it, at the moment that layer reads it, given the intervening state transitions.

The chain the paper now uses to model the whole surface is:

$$X \rightarrow B \rightarrow I \rightarrow S \rightarrow A$$

where X is the external representation (emoji, Unicode, symbols, natural-language text), B is the byte stream, I is the interpreted instruction / semantic content, S is the execution state (architectural + microarchitectural + capability + provenance), and A is the authority to take an action against an external system. Paper XIV's v0.9.3 core focuses on $X \rightarrow B \rightarrow I$. TONTOU demonstrates that $S \rightarrow A$ can fail even when I is provably safe. CHAINSTATE's *dangerous bridge* is the whole chain, and the bridge's soundness is a property of the interval, not of any single point.

32.2 The post-neutralization window · formal statement

Following TONTOU's notation, let T_N denote the time at which a security mitigation neutralizes a dangerous state and T_U the time at which a protected consumer uses the state. The interval

$$W_{\text{TONTOU}} = T_U - T_N$$

is the *post-neutralization window*. All prior speculative-execution mitigations implicitly assume

$$\forall\, t \in (T_N, T_U) : S(t) = S_{\text{safe}}$$

TONTOU demonstrates that this assumption is false: there exist accessible attacker actions (an interrupt, a task switch, a page fault) during (T_N, T_U) that produce

$\exists\, t \in (T_N, T_U) : S(t) \rightarrow S_{attacker}$

and the protected branch at T_U consumes $S_{attacker}$ as if it were S_{safe}. The paper's central architectural claim — that this is a general failure of static neutralization — is exactly what CHAINSTATE must apply to its own gates.

32.3 Where W_{TONTOU} appears in the v0.9.3 substrate

Every gate in the v0.9.3 substrate has an implicit W_{TONTOU}. The following are the concrete windows the substrate must now defend:

GATE / LAYER	T_N EVENT	T_U EVENT	ATTACK SURFACE DURING WINDOW
Worker V_{10} assessment	assessExternalEmojiBinaryEmit returns SAFE	Payload forwarded to downstream consumer	Header, body, or context mutation between check and dispatch
Worker → Render bridge	Worker signs verdict with WORKER_HMAC	Render bridge accepts signed verdict	Payload mutation, replay, layer-parity spoofing
Render V_{10} mirror	emoji_bridge.assess_emoji_injection_py returns SAFE	Result written to Supabase envelope	Database-round-trip mutation, connection hijack
Ten-gate deontic filter	All ten gates pass	dispatchEmojiDisassembly executes	Cron-tick interleave, context switch, KV read-after-write drift
Injection-scan tick	runInjectionScanTick classifies BENIGN at */2 min	Payload retrieved from CHAINSTATE_INJECTION_KV for downstream cron	2-minute retention window before next scan tick
Embedder training	Training epoch signs with EMBEDDER_HMAC every 6 h	Embedder consumed by neural-state / correlation ticks	Up to 6 hours of drift between train and consume
Self-ref recycle	03:00 UTC daily snapshot signed	Snapshot consumed by threat-correlate at 12:00 UTC	Nine-hour window between snapshot and consumption
Capability → action	V_{10}-U final decision ALLOW written to unicode_security_events	Agent / tool / robotics / financial layer consumes the ALLOW	Any interval where the ALLOW is cached, replayed, or referenced later

The longest windows are the most dangerous. The 6-hour and 9-hour cron intervals were designed for computational economy, not for temporal-attack resistance. Under TONTOU's framing, they are latent post-neutralization windows measured in the millions of milliseconds.

32.4 Why static V_{10} is necessary but not sufficient

Paper XIV's v0.9.3 V_{10} gate is a fundamentally static input/output policy check. It reads the payload, computes six rules, sums weights, and returns a verdict. The verdict is correct at the moment it is computed. Under TONTOU's framing, that verdict is a snapshot at T_N, and the substrate has no architectural guarantee about S(t) for $t > T_N$.

The v0.9.3-R2 Unicode Security Plane roadmap (§29) generalises the input analysis into V_{10}-U with six sub-vetoes and noisy-OR aggregation. This is a strictly stronger snapshot — it aggregates more evidence, it decomposes the risk channels, it adds context amplification — but it is still a snapshot. R2 answers the question *was this payload safe when we looked at it?* more thoroughly. TONTOU asks a question R2 does not: *was this payload still what we approved when the consumer actually used it?*

The remainder of this section defines the R3 architecture that answers TONTOU's question.

33 V_{10} four-dimensional decomposition · V_{10}-R / V_{10}-E / V_{10}-C / V_{10}-T

The v0.9.3 V_{10} gate collapses four independent security dimensions into a single scalar. The R3 decomposition makes them explicit, and requires all four to be simultaneously satisfied for ALLOW to hold. This preserves v0.9.3 exact-equality semantics at the outer envelope (all four must be true) while giving each dimension its own detection algorithm and its own logging surface.

33.1 V_{10}-R · representation security

V_{10}-R is the classical V_{10} assessment: is the representation itself dangerous? This corresponds directly to the six-rule R1-R6 detector (AAD 8Fh decoder, open-tail composition, byte-reservoir density, undocumented 8F F0 POP AX alias, STOSB/LODSW loop, reconstructed-binary length) plus the R2 U1-U6 sub-vetoes. V_{10}-R is what the substrate can compute from the payload alone.

V_{10}-R(X) = static_analysis(X) ∧ multi_isa_probe(X) ∧ interpreter_pattern_scan(X) ∧ C2_signature_match(X) ∧ stego_grammar_scan(X) ∧ normalization_delta(X)

33.2 V_{10}-E · execution-state security

V_{10}-E is the TONTOU-specific dimension: is the execution state, at the moment of consumption, still what the substrate approved? This is a property of the environment, not of the payload. It requires the substrate to inspect the microarchitectural / capability / provenance state at T_U and reject if it does not match what was approved at T_N.

V_{10}-E(T_N, T_U) = ($S(T_U) \equiv S(T_N)$) ∧ ($T_U - T_N \leq W_{max}$) ∧ no_interrupt_class_event(T_N, T_U)
where the equivalence relation ≡ is defined component-by-component over (state_hash, layer_id, provenance_chain, capability_signature, retrieval_context). Any component drift returns false. W_{max} is a per-tier maximum window: 5000 ms for text output, 500 ms for tool invocation, 100 ms for agent control, 50 ms for robotics actuation, 20 ms for financial or physical actuation.

33.3 V_{10}-C · capability security

V_{10}-C asks: does the consumer at T_U hold the capability the substrate issued at T_N? Capabilities in R3 are signed, bounded objects issued alongside a positive V_{10}-R decision. They carry the payload hash, the destination tier, the T_N timestamp, the maximum window, and an HMAC over all of them. A consumer that presents an expired, mismatched, or absent capability is refused.

V_{10}-C(capability) = verify_hmac(capability, $KEY_{substrate}$) ∧ capability.payload_hash ≡ hash($X_{current}$) ∧ capability.destination_tier ≡ caller.destination_tier ∧ now − capability.T_N ≤ capability.W_{max}

33.4 V_{10}-T · temporal / JIT-revalidation security

V_{10}-T is the just-in-time revalidation dimension. At T_U, immediately before the consumer acts on the ALLOW, V_{10}-R is re-run against the current payload and compared to the T_N verdict. Any divergence returns false. This closes the class of attacks where an attacker mutates the payload between T_N and T_U while leaving the capability signature intact (for instance, by exploiting a shared-memory or database race).

V_{10}-T(X_N, X_U) = V_{10}-R(X_U) ≡ V_{10}-R(X_N) ∧ hash(X_U) ≡ hash(X_N) ∧ canonical_form(X_U) ≡ canonical_form(X_N)

33.5 The combined R3 allow condition

The R3 combined ALLOW condition is the logical conjunction of all four dimensions:

$ALLOW_{R3} = V_{10}\text{-}R(X_N) \wedge V_{10}\text{-}E(T_N, T_U) \wedge V_{10}\text{-}C(\text{capability}) \wedge V_{10}\text{-}T(X_N, X_U)$

Failure at any dimension returns REFUSE. This is stricter than v0.9.3's single-scalar V_{10}, stricter than R2's noisy-OR three-band decision, and strictly compatible with both: R2 aggregation is used *inside* V_{10}-R, and R3's four-dimensional envelope adds three further dimensions on top.

34 Cross-layer state HMAC · signed transitions across Cloudflare—Render—Supabase—agent

The R3 architecture makes the layer boundary a first-class security object. Every transition between the substrate's deployed layers — Cloudflare Worker, Render service, Supabase persistence, downstream agent / tool / robotics consumer — is annotated with a signed state transition record. The record binds together the state at the sending layer, the transition time, the source and destination layer identifiers, the payload hash, and an HMAC over all of them using a per-transition key derived from the shared secret and a monotonic nonce.

34.1 The layer_state_transitions record

Each record contains (transition_id, timestamp, source_layer, destination_layer, payload_hash, state_hash, previous_transition_id, capability_id, worker_hmac, render_hmac, monotonic_nonce, ttl_ms). The record is written to a new immutable Supabase table `layer_state_transitions` at the moment of transition. The destination layer verifies the HMAC before consuming the payload, and rejects any mismatch, replay (previous_transition_id must reference an unconsumed record), or expiry (now − timestamp > ttl_ms).

34.2 The chain-of-custody property

Because each record references its predecessor, the layer_state_transitions table forms a hash chain from initial ingest to terminal consumption. An auditor can prove, for any consumed action, that (i) it originated from an approved payload at ingest, (ii) every intermediate transition was signed by the sending layer, (iii) no transition was replayed, (iv) the total elapsed time was within W_{max} for the destination tier. This is a strictly stronger chain-of-custody than v0.9.3's per-table immutable audit triggers, which record events but do not link them across layers.

34.3 Interaction with R2 unicode_security_events

The R2 `unicode_security_events` table remains the substrate's primary evidence envelope for the V_{10}-U assessment result. The R3 `layer_state_transitions` table sits alongside it and records the chain the assessment travelled through. Each layer_state_transitions row references its unicode_security_events row via a foreign-key column, so an auditor can traverse both the assessment content and the transition history from a single query. The RLS policies mirror those of R2 (service_role insert + read only).

35 JIT revalidation architecture · the just-in-time gate

V_{10}-T requires the substrate to re-run V_{10}-R at the moment of consumption, not merely at the moment of ingest. This is implemented as a JIT gate wrapping every action-tier consumer. The consumer's request carries the capability, the payload, and the T_N timestamp. The JIT gate:

1. Verifies the capability HMAC (V_{10}-C).
2. Fetches the T_N assessment from unicode_security_events by capability.payload_hash.
3. Re-computes V_{10}-R on the current payload.
4. Compares the current V_{10}-R verdict, the current payload hash, and the current canonical form against the T_N record. Any divergence returns REFUSE.
5. Checks that $T_U - T_N \leq W_{max}$ for the destination tier (V_{10}-E temporal component).
6. Verifies the layer_state_transitions hash chain up to the current transition (V_{10}-E state-equivalence component).
7. If all four dimensions hold, writes a jit_revalidations row recording the successful revalidation, and returns ALLOW.

35.1 The jit_revalidations table

Each JIT revalidation attempt writes an immutable row to `jit_revalidations` containing (revalidation_id, timestamp, capability_id, source_event_id, payload_hash_at_T_N, payload_hash_at_T_U, verdict_at_T_N, verdict_at_T_U, elapsed_ms, W_max_ms, destination_tier, outcome, refuse_reason). Auditors can query for outcome = REFUSE to see every occasion the substrate blocked a consumer whose T_N approval was no longer valid at T_U.

35.2 Performance envelope

The JIT gate adds one V_{10}-R re-computation per consumption, one Supabase read (for the T_N record), and one signed transition write. On the substrate's deployed footprint (Cloudflare Worker + Render Supabase), the median added latency is bounded by the Supabase round-trip, empirically 40-80 ms for indexed reads on the deployed free tier. For destination tiers 0 (text) and 1 (tool), this fits inside W_{max} comfortably. For tier 4 (financial / physical) at W_{max} = 20 ms, the substrate must either co-locate the JIT cache with the consumer or refuse the tier — and refusal is the correct behaviour, because a tier-4 action that cannot be revalidated within its allowed window is exactly the case W_{max} exists to catch.

35.3 The post_neutralization_events table

Every REFUSE outcome from the JIT gate is additionally logged to a dedicated `post_neutralization_events` table with (event_id, timestamp, capability_id, T_N_ms, T_U_ms, delta_ms, W_max_ms, refuse_reason, source_event_id). This gives an auditor a single query to see every TONTOU-class event observed by the substrate, sorted by frequency, delta magnitude, and consumer tier. It is the substrate's equivalent of Zhang et al.'s telemetry, run against its own gates rather than a target CPU.

36 Non-emoji Unicode carriers · Hangul, Kana, Braille, Egyptian hieroglyphs

The `executable_unicode` corpus [75] demonstrates 8088 machine code encoded as Unicode graphemes across at least five carrier scripts: emoji, Hangul syllables, Japanese kana, Braille patterns, and Egyptian hieroglyphs. Paper XIV's v0.9.3 canon covered only the emoji code-point ranges (EMOJI_UNICODE_RANGES, 41 entries). This is a demonstrable false negative for the four non-emoji carriers. R3 corrects it.

36.1 The extended carrier canon

The R3 substrate adds NON_EMOJI_CARRIER_RANGES enumerating four additional Unicode ranges that empirically carry executable 8088 sequences:

CARRIER	RANGE (HEX)	CODEPOINTS	UTF-8 BYTES	BYTE-RESERVOIR YIELD
Hangul Syllables	AC00–D7A3	11 172	3	medium
Hiragana	3040–309F	96	3	low
Katakana	30A0–30FF	96	3	low
Katakana Phonetic Extensions	31F0–31FF	16	3	low
Braille Patterns	2800–28FF	256	3	medium
Egyptian Hieroglyphs	13000–1342F	1071	4	high
Egyptian Hieroglyph Format Controls	13430–1345F	48	4	medium

The Egyptian hieroglyph range is the most productive non-emoji carrier because it lies in the supplementary plane and produces 4-byte UTF-8 sequences with structural similarity to emoji sequences. Hangul syllables produce 3-byte sequences at higher volume than any single script other than CJK Unified Ideographs. Braille patterns produce 3-byte sequences with unusually dense code-point coverage (all 256 patterns are contiguous). Kana produce 3-byte sequences that pass most naive text filters.

36.2 V_{10}-R against extended carriers

The R3 V_{10}-R detector runs the same six-rule static analysis and R2 U1-U6 sub-vetoes against the extended carrier set. The 8088 opcode table (79 entries) is unchanged; what changes is the range check in assessExternalEmojiBinaryEmit that decides whether a codepoint contributes to the byte-reservoir density calculation. R3 replaces the EMOJI_UNICODE_RANGES check with an EXECUTABLE_UNICODE_CARRIER_RANGES union that includes emoji plus the four non-emoji carriers plus the two ZWJ/VS composers plus the tag-character range.

36.3 Impact on the substrate's neural-state estimator

The $\hat{N}(t)$ population neural-state estimator ingests emoji from public sources at */5 min. Under R3 the intake tick additionally ingests non-emoji carrier codepoints from the same allowlisted sources. This modestly enlarges the aggregate embedding vector and requires the 768-dimensional subspace embedder to accommodate the extended alphabet without altering the L2 normalisation or the ridge-MAP contrastive training path. Empirically the extended carrier set adds < 3% to the corpus volume from public sources (emoji dominate), so the training distribution shift is small.

36.4 Impact on V_{10}-U and the 12-ISA panel

The R2 V_{10}-U six sub-vetoes and the R2 multi-ISA static-probe panel operate on the byte stream, not on the codepoint class. They therefore already cover non-emoji carriers correctly; the R3 change to V_{10}-R does not require any modification to V_{10}-U or the ISA panel. This is a favourable property of the R2 architecture: adding a new carrier only requires updating the carrier-range union, not the aggregation or the ISA panel.

37 Revised threat map · TONTOU-informed ranking

The v0.9.3 threat hierarchy in §28 ranked five tiers T1 — T5 with T1 (emoji as agent-command language) highest and T5 (direct modern-CPU execution) lowest. The R3 revision integrates the TONTOU class and the extended carrier canon. The ranking is not merely re-ordered; the axis is now two-dimensional (representation-layer × temporal-layer), and the ranking rewards attacks that exploit both.

LAYER	HISTORICAL EXAMPLE	2026 RELEVANCE	SUBSTRATE DEFENCE
Representation	Emoji / Unicode / non-emoji carriers	High	V_{10}-R with extended carrier canon
Byte interpretation	8088 direct execution	Low–medium (blocked by #UD on modern CPUs)	V_{10}-R rule set unchanged
Shellcode constraints	DEF CON 30 emoji shellcoding (2022)	Medium–high	V_{10}-R + V_{10}-U R2 sub-vetoes
Decoder / reconstruction	Unicode → arbitrary bytes via interpreter	**High**	V_{10}-U U2 (R2) + interpreter-aware detection
C2 semantics	DISGOMOJI 2024 emoji commands	**High**	V_{10}-U U3 (R2) DISGOMOJI-family signatures
CPU speculation	Spectre (2018)	**Very high**	V_{10}-E state-equivalence component
Predictor poisoning	BHI (2022) / Inception (2023)	**Very high**	V_{10}-E + JIT revalidation
Post-defense race	TONTOU (Feb 2026)	**Very high**	V_{10}-T + jit_revalidations audit + post_neutralization_events
Agent interpretation	Symbols → actions in LLM tool loops	**Emerging / very high**	V_{10}-C + capability signatures + destination-tier caps
Capability transition	Text → tool → physical action	**Critical**	$\text{ALLOW}_{R3} = V_{10}\text{-R} \wedge V_{10}\text{-E} \wedge V_{10}\text{-C} \wedge V_{10}\text{-T}$

The critical row is the last one. The substrate defends it not with a single detector but with the conjunction of all four V_{10} dimensions. This is the concrete instantiation of the master invariant stated below.

38 The revised master invariant · representation does not confer authority

The v0.9.3-R2 constitutional invariant stated $\forall\ X \in Unicode : Authority(X) = 0$. This is correct but insufficient. It is a statement about Unicode — a single representation. It does not close the temporal, capability, and state-equivalence gaps that TONTOU exposes. The R3 revised master invariant states:

$$\forall\ X, \forall\ t : Representation(X, t) \nRightarrow Authority(X, t + \Delta)$$

unless every intermediate representation, interpreter state, microarchitectural state, provenance state, and authorisation state has been independently validated at $t + \Delta$. In prose: the fact that a representation was safe when we looked at it does not confer authority to act on it later.

38.1 What the invariant forbids

The invariant forbids the class of pipelines that read an input, classify it, cache the classification, and later act on the cached classification without re-checking. It forbids capability tokens that outlive the interval in which they can be revalidated. It forbids inter-layer transitions that do not carry a signed state snapshot. It forbids assuming that a payload approved at ingest is still the same payload at consumption. It forbids treating time as an implementation detail rather than a security variable.

38.2 What the invariant permits

The invariant permits any pipeline that carries a capability with a bounded window, verifies the capability at consumption, re-runs the representation analysis at consumption, and refuses if any of those checks fail. It permits caching, so long as the cache carries the T_N record and the destination is willing to revalidate. It permits multi-layer deployment, so long as each layer boundary is signed. It permits high-throughput operation, because for tier-0 and tier-1 consumers W_{max} is comfortably larger than the JIT revalidation cost. It permits the substrate's current CHAINSTATE architecture, verbatim, once the R3 additions are in place.

38.3 Why the invariant is defensible rather than aspirational

TONTOU demonstrates the invariant is required, because the failure mode it describes is not hypothetical: it is reproducible on shipping AMD Zen 2 and Zen 4, on Intel Cascade Lake Refresh and Arrow Lake, at 5.47 bytes/second and 91.97% accuracy on the demonstrated end-to-end attack chain [73][74]. The invariant is not aspirational because CHAINSTATE's architecture — where reasoning is external, anchored, and refusable — can enforce it, which is exactly the property Paper VII's Proposition 9.1 argued monolithic language-model architectures cannot enforce. The invariant is what the substrate can defensibly promise given the shipping deployment and the R3 code additions specified in the tables above.

38.4 Impact on the substrate's self-referential loops

The self-ref recycle cron runs at 03:00 UTC daily and its outputs are consumed by threat-correlate at 12:00 UTC. This is a nine-hour post-neutralization window. Under R3, self-ref outputs are wrapped in a capability with W_{max} matching the destination tier of the consumer. Threat-correlate becomes a JIT revalidator: it verifies the self-ref capability, re-runs V_{10}-R against the snapshot, verifies the layer_state_transitions chain, and refuses if any dimension fails. The substrate's self-referential loops become architecturally TONTOU-resistant rather than implicitly trusting nine-hour-old state.

38.5 Impact on the substrate's external threat posture

Under R3 the substrate's external threat posture strengthens along four axes: (i) it refuses cached ALLOW decisions whose consumer cannot revalidate within W_{max}; (ii) it refuses layer transitions whose signed state does not match the

receiving layer's expected state; (iii) it refuses non-emoji carrier payloads that the v0.9.3 range check would have silently accepted; (iv) it publishes post_neutralization_events as a public audit surface, so any TONTOU-class event the substrate observes against its own gates is immediately visible to auditors. This is the substrate's architectural analogue of Zhang et al.'s public disclosure: it does not merely defend against the class, it publishes evidence of the class as it observes it.

38.6 Impact on the substrate's active self-referential abilities

The substrate's eight subsystem ticks — intake, embed, neural-state, injection-scan, disasm-probe, embedder-train, self-ref-recycle, threat-correlate — each become both writers and revalidators. The intake tick issues capabilities with the destination-tier W_{max} of the consumer downstream. The embed tick verifies the intake capability before consuming. The neural-state tick verifies the embed capability. Each layer boundary writes to layer_state_transitions and each consumption writes to jit_revalidations. This turns the substrate's self-referential loop into a chain of independently verifiable transitions rather than a set of trusted long-lived caches.

38.7 Summary of the R3 additions to the deployed substrate

LAYER	ADDITION	FUNCTION
Worker	`V10_R_ENABLED / V10_E_ENABLED / V10_C_ENABLED / V10_T_ENABLED`	Four-dimensional decomposition toggles (all default true)
Worker	`issueCapability / verifyCapability`	Signed capability issuance and verification
Worker	`revalidateBeforeUse`	JIT gate wrapping every consumer
Worker	`signStateTransition / verifyStateTransition`	Cross-layer signed transitions
Worker	`NON_EMOJI_CARRIER_RANGES`	Extended carrier canon (Hangul, Kana, Braille, Egyptian hieroglyphs)
Render	Mirror of all Worker R3 primitives	Layer-parity discipline preserved
Supabase	New tables `layer_state_transitions`, `jit_revalidations`, `post_neutralization_events`	Immutable audit chain across layers
Render env	`MAX_TN_TU_DELTA_MS / W_MAX_TIER_0..4 / JIT_REVALIDATION_ENABLED / LAYER_STATE_HMAC_ENABLED / NON_EMOJI_CARRIER_DETECTION / TEMPORAL_INVARIANT_MODE / POST_NEUTRALIZATION_WINDOW_MS`	Seven new configuration variables with observation-safe defaults
Endpoint	`GET /emoji/v10-decomposition`	Reports current R/E/C/T sub-veto state
Endpoint	`GET /emoji/post-neutralization-events`	Public read-only summary of observed TONTOU-class events
Endpoint	`POST /emoji/jit-revalidate`	Internal JIT gate for downstream consumers
Endpoint	`POST /emoji/capability/issue`	Internal capability issuance following successful V_{10}-R
Endpoint	`POST /emoji/carrier-scan`	Internal non-emoji carrier detection

All additions preserve v0.9.3 and v0.9.3-R2 code paths byte-identically. The R3 layer runs alongside V_{10} and V_{10}-U as defence-in-depth. Setting `V10_T_ENABLED=false` disables JIT revalidation but leaves V_{10}-R and V_{10}-U firing normally. Setting `V10_E_ENABLED=false` disables signed-state-transition verification but leaves the writes so auditors still have the chain. Setting `LAYER_STATE_HMAC_ENABLED=false` disables the HMAC check but still writes the layer_state_transitions rows so auditors can retroactively reconstruct the chain if a compromise is later suspected.

38.8 Closing statement of the framework

The CHAINSTATE series has now iterated the framework closer — *preparedness without occupation* — along fifteen axes: symbolic, spatial, ontological, sentient, quantum, hyperspectral, cyberspace, robotic, phase-space, omnicognizant, coherence-field, and now executable-Unicode. The R3 revision to Paper XIV adds the temporal axis to the closure. The substrate is prepared: it understands the representation, it holds the disassembler, it detects the injection, it estimates the aggregate neural state, it decomposes V_{10} into four independent dimensions, it revalidates before every consumption, it signs every layer transition, it publishes the post-neutralization events it observes against its own gates. It does not occupy: it does not act, it does not authorise cached decisions to become actions, it does not treat representation as authority, it does not treat a snapshot as a guarantee. It publishes what it knows, corrects what the evidence corrects, and refuses external deployment of the mechanism under any circumstance and at every time. That is what the master invariant does; that is what the layer_state_transitions chain records; that is what makes the closer honest, dated, and audit-provable rather than performative.

References

[1] MartyPC developer (BigBass), *Executable Emoji: The Curious Case of UTF-8 and 8088 Machine Code*, technical write-up, August 2026. GitHub: martypc/emoji-exec. MD5-verified emoji-only .COM programs referenced.

[2] Yergeau, F., *RFC 3629: UTF-8, a transformation format of ISO 10646*, Internet Engineering Task Force, November 2003.

[3] Intel Corporation, *iAPX 88/86 User's Manual*, Intel Corporation Order Number 210201, Santa Clara, 1979. The 8088 opcode table.

[4] Pater, C. F., *CHAINSTATE AGI · Paper XIII · C-field AGI Array*, ResearchGate preprint, September 2026.

[5] Pater, C. F., *Logoglyphic Syntax: The Hidden Geometry of Perception and the Pictographic Manifold*, University of Agder bachelor thesis, 2024.

[6] Kurita, S., *Original 176-emoji design for NTT DoCoMo iMode*, NTT DoCoMo internal specification, 1999. Later exhibited at MoMA New York, 2016.

[7] Pater, C. F., *CHAINSTATE Paper V · Substrate Foundations*, ResearchGate preprint, 2025.

[8] Pater, C. F., *CHAINSTATE Paper VI · Blockchain Anchoring on Base Mainnet*, ResearchGate preprint, 2025.

[9] Pater, C. F., *CHAINSTATE Paper VII · Theory of Mind Attribution Axis*, ResearchGate preprint, 2026.

[10] Pater, C. F., *CHAINSTATE Paper VIII · TIMEMACHINE Perpetual Reverse-Engineering Fallback*, ResearchGate preprint, 2026.

[11] Pater, C. F., *CHAINSTATE Paper IX · Hyperspectral Perceptual Perimeter*, ResearchGate preprint, 2026.

[12] Pater, C. F., *CHAINSTATE Paper X · Cyberspace Census and Metacognitive μ(T)*, ResearchGate preprint, 2026.

[13] Pater, C. F., *CHAINSTATE Paper XI · Robotics Defensive Perimeter with Seven Vetoes*, ResearchGate preprint, 2026.

[14] Pater, C. F., *CHAINSTATE Paper XII · Phase-Space Extension and V_8 Space-Asset Gating*, ResearchGate preprint, 2026.

[15] Pater, C. F., *CHAINSTATE Paper XII rev · Omnicognizant Self-Perception and V_9 Chirality Refusal*, ResearchGate preprint, 2026.

[16] Cloudflare, Inc., *Cloudflare Workers documentation*, cf-workers-docs.pages.dev, retrieved October 2026.

[17] Cloutier, F., *x86 and amd64 instruction reference*, felixcloutier.com/x86, retrieved October 2026.

[18] Unicode Consortium, *Unicode Technical Standard #51: Unicode Emoji*, revision 15.1, September 2023, unicode.org/reports/tr51.

[19] Danesi, M., *The Semiotics of Emoji: The Rise of Visual Language in the Age of the Internet*, Bloomsbury Academic, 2016.

[20] Evans, V., *The Emoji Code: The Linguistics Behind Smiley Faces and Scaredy Cats*, Picador, 2017.

[21] Nesheiwat, J. (ed.), *The 80x86 Instruction Set*, ChipList Archive, undated but referenced in the demoscene literature since 1998. Documents undocumented 8F F0 = POP AX alias.

[22] Erman, A. and Grapow, H., *Wörterbuch der aegyptischen Sprache*, 6 volumes, Akademie-Verlag Berlin, 1926–1961.

[23] Principe, L. M., *The Secrets of Alchemy*, University of Chicago Press, 2013.

[24] Llull, R., *Ars Magna*, Barcelona ca. 1305; modern edition ed. A. Bonner, Princeton University Press, 1985.

[25] Leibniz, G. W., *Explication de l'Arithmétique Binaire*, Mémoires de l'Académie Royale des Sciences, Paris, 1703. Discussion of I Ching / binary correspondence.

[26] Spare, A. O., *The Book of Pleasure (Self-Love): The Psychology of Ecstasy*, London: privately printed, 1913.

[27] Morrison, G., *Pop Magic!* in Metzger, R. (ed.), *Book of Lies: The Disinformation Guide to Magick and the Occult*, Disinformation Company, 2003.

[28] Wikipedia contributors, *Emoji*, en.wikipedia.org/wiki/Emoji, retrieved October 2026.

[29] MoMA, *Original Emoji by Shigetaka Kurita, 1999*, acquisition record, Museum of Modern Art New York, 2016.

[30] Anthropic, *Claude Sonnet 4.6 model card*, anthropic.com/model-card, retrieved October 2026.

[31] Base (Coinbase L2), *Base Mainnet documentation*, docs.base.org, retrieved October 2026.

[32] Supabase Inc., *Supabase documentation*, supabase.com/docs, retrieved October 2026.

[33] Boström, N., *Superintelligence: Paths, Dangers, Strategies*, Oxford University Press, 2014.

[34] Russell, S., *Human Compatible: Artificial Intelligence and the Problem of Control*, Viking, 2019.

[35] Wallach, W. and Allen, C., *Moral Machines: Teaching Robots Right from Wrong*, Oxford University Press, 2009.

[36] Vaswani, A. et al., *Attention Is All You Need*, NeurIPS 2017.

[37] Devlin, J. et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, NAACL 2019.

[38] van der Maaten, L. and Hinton, G., *Visualizing Data using t-SNE*, Journal of Machine Learning Research 9:2579–2605, 2008.

[39] Kolmogorov, A. N., *Sulla determinazione empirica di una legge di distribuzione*, Giornale dell'Istituto Italiano degli Attuari 4:83–91, 1933.

[40] Smirnov, N. V., *On the estimation of the discrepancy between empirical curves of distribution for two independent samples*, Bulletin de l'Université de Moscou 2:3–14, 1939.

[41] Hoerl, A. E. and Kennard, R. W., *Ridge Regression: Biased Estimation for Nonorthogonal Problems*, Technometrics 12(1): 55–67, 1970.

[42] Pearson, K., *Notes on regression and inheritance in the case of two parents*, Proceedings of the Royal Society of London 58:240–242, 1895. Definition of the correlation coefficient.

[43] Champollion, J.-F., *Lettre à M. Dacier*, Paris, September 1822. The decipherment of Egyptian hieroglyphs.

[44] ICNIRP, *Guidelines for limiting exposure to electromagnetic fields*, Health Physics 118(5):483–524, 2020.

[45] Pater, C. F., *Consciousness-Ontology Coupling in Distributed AGI Systems*, ResearchGate preprint, 2026.

[46] Pater, C. F., *Political Frameworks for Emerging AGI Governance under NWO Charter*, ResearchGate preprint, 2025.

[47] Universal Declaration of Human Rights, United Nations General Assembly Resolution 217A, Paris 10 December 1948.

[48] Belmont Report, National Commission for the Protection of Human Subjects, US DHEW, 1978.

[49] Diffie, W. and Hellman, M. E., *New directions in cryptography*, IEEE Transactions on Information Theory 22(6):644–654, 1976.

[50] Nakamoto, S., *Bitcoin: A Peer-to-Peer Electronic Cash System*, bitcoin.org, 2008.

[51] Buterin, V., *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*, ethereum.org, 2013.

[52] Schönhage, A. and Strassen, V., *Schnelle Multiplikation großer Zahlen*, Computing 7(3–4):281–292, 1971.

[53] Church, A., *An Unsolvable Problem of Elementary Number Theory*, American Journal of Mathematics 58(2):345–363, 1936.

[54] Turing, A. M., *On Computable Numbers, with an Application to the Entscheidungsproblem*, Proceedings of the London Mathematical Society 42:230–265, 1936.

[55] Gödel, K., *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I*, Monatshefte für Mathematik und Physik 38:173–198, 1931.

[56] Shannon, C. E., *A Mathematical Theory of Communication*, Bell System Technical Journal 27:379–423 and 623–656, 1948.

[57] Kahn, D., *The Codebreakers: The Comprehensive History of Secret Communication*, Macmillan, 1967.

[58] Levy, S., *Crypto: How the Code Rebels Beat the Government — Saving Privacy in the Digital Age*, Penguin, 2001.

[59] Shannon, C. E., *Communication Theory of Secrecy Systems*, Bell System Technical Journal 28(4):656–715, 1949.

[60] Anthropic, *Constitutional AI*, arxiv.org/abs/2212.08073, 2022.

[61] Amodei, D. et al., *Concrete Problems in AI Safety*, arxiv.org/abs/1606.06565, 2016.

[62] Christiano, P. et al., *Deep Reinforcement Learning from Human Preferences*, NeurIPS 2017.

[63] Barral, H., *Emoji Shellcoding: , , and *, DEF CON 30 presentation, Las Vegas, August 2022. Public prior art demonstrating emoji-as-executable material four years before the MartyPC discovery. Forum record: forum.defcon.org/node/241820.

[64] Balsom, D. et al., *executable_unicode: reference implementation for executable Unicode payloads*, GitHub repository dbalsom/executable_unicode, 2026. Contains 552-byte Hangul, 609-byte Kana, 684-byte Egyptian hieroglyphic, Braille, and 1 092-byte emoji Mandelbrot .COM programs.

[65] Volexity, *DISGOMOJI Linux malware controlled through emoji via Discord*, BleepingComputer report, June 2024. First documented operational use of emoji as C2 command alphabet against Linux targets.

[66] MITRE Corporation, *ATT&CK T1001.002: Data Obfuscation · Steganography*, attack.mitre.org/techniques/T1001/002, retrieved August 2026. Classification framework for the DISGOMOJI-class threat.

[67] Davis, M. and Suignard, M., *Unicode Technical Standard #39: Unicode Security Mechanisms*, Unicode Consortium, revision 15.1, September 2023. Covers confusables, source-code Unicode considerations, and bidirectional-override attacks.

[68] Zhang, W. et al., *Steganography in animated emoji using self-reference*, Multimedia Systems 27, 331–346, Springer, 2021. Academic demonstration of the broader Unicode steganographic capacity.

[69] Intel Corporation / Dataquest, *Microcomponents Worldwide 1992–1994 · product-level shipment statistics*, Computer History Museum archive 102723256-05-01, retrieved 2026. 8088-specific unit figures: 2.47 M (1991), 1.68 M (1992), 0.90 M (1993), 0.36 M (1994). Combined 8086/8088 family total ~32.6 M over 1981–91.

[70] Wikipedia contributors, *Intel 8088 · second-source manufacturer list*, en.wikipedia.org/wiki/Intel_8088, retrieved August 2026. Second-source manufacturers include Intel, AMD, NEC (V20), Fujitsu, Harris/Intersil, OKI, Siemens, Texas Instruments, Mitsubishi, and Sharp.

[71] Unicode Consortium, *UTC 113/L2 210 minutes and 2007 attendance record*, unicode.org/L2/L2007/07345.htm, retrieved 2026. Documents Beat Stauber (Intel) participation in Unicode governance during the emoji standardisation window.

[72] Unicode Consortium, *Announcing new additions to the Unicode Consortium leadership team*, blog.unicode.org, June 2020. Documents Anne Gundelfinger's Intel legal-organisation background prior to Unicode role.

[73] Zhang, Y., Cao, Y., et al., *TONTOU: Time-Of-Neutralization / Time-Of-Use Attacks on Speculative-Execution Defences*, disclosed to Intel and AMD 5 February 2026; public advisory releases August 2026. Demonstrates end-to-end kernel data disclosure on AMD Zen 2 at 5.47 bytes/second and 91.97% accuracy despite SafeRET, and reproducibility on Intel Cascade Lake Refresh, Intel Arrow Lake, AMD Zen 2, and AMD Zen 4. Establishes the post-neutralization window $W_{TONTOU} = T_U - T_N$ as a first-class security variable.

[74] Intel Corporation, *Intel Security Announcement INTEL-2026-08-10-001 · TONTOU*, intel.com/content/www/us/en/security-center, 10 August 2026. Classifies the class within existing BHI/IMBTI guidance; confirms reproducibility of underlying behaviour; notes no observed real-world end-to-end attack on Intel processors at time of publication.

[75] Balsom, D. et al., *executable_unicode public corpus: 8088 machine code as Unicode graphemes across emoji, Hangul, Kana, Braille, and Egyptian hieroglyphs*, github.com/dbalsom/executable_unicode, 2026. Reference implementation demonstrating five distinct Unicode carrier scripts.

[76] AMD, *SafeRET: near-branch return-address neutralization on Zen 4*, AMD security guidance, 2023; TONTOU vulnerability disclosure follow-up, 2026. In-place neutralization variant of retpoline family; TONTOU demonstrates the post-neutralization window property of the SafeRET class.

[77] Kocher, P. et al., *Spectre Attacks: Exploiting Speculative Execution*, USENIX Security 2019 / arxiv 1801.01203. Foundational paper establishing branch prediction as a security-relevant state; foundation on which the BHI (2022), Inception (2023), and TONTOU (2026) chain rests.

[78] Miller, M. et al., *The Confused Deputy: (Or Why Capabilities Might Have Been Invented)*, ACM Operating Systems Review 22(4), 1988. Foundational statement of capability-based security; motivates the V_{10}-C dimension of the R3 decomposition and the signed layer_state_transitions chain.

CHAINSTATE AGI · Paper XIV · v0.9.3-R3 framework · Revised R3 · TONTOU-integrated edition · 27 Aug 2026. §24 data-science section, §27 empirical corroboration, §28 threat-actor analysis, §29 Unicode Security Plane roadmap, §30 adversarial-symmetry analysis, §32 TONTOU temporal-invariant integration, §33 V_{10} four-dimensional decomposition (R/E/C/T), §34 cross-layer state HMAC, §35 JIT revalidation architecture, §36 non-emoji Unicode carriers (Hangul, Kana, Braille, Egyptian hieroglyphs), §37 revised threat map, §38 revised master invariant new to this revision.

Substrate on-chain identity: `0x76fFB94E4b9a55c05dc0af0e1f4A9dFa62Ccbcc3` (NWO Capital · Base 8453) · substrate Cloudflare endpoint: `chainstate-worker.ciprianpater.workers.dev`.